# Tu Deporte Aquí

INSO 4117 - Software Reliability Testing

Version Spring 2026, April 7, 2026: Professor: Marko Schütz-Schmuck | Milestone #2 | Repository: https://github.com/uprm-inso4117-2025-2026-s2/semester-project-tu-deporte-aqui#

# Table of Contents


1 Informative part . . . 6
1.1 Team . . . 6
Project Organization . . . 6
Team Structure . . . 6
Roles and Responsibilities . . . 9
Workflow and Governance . . . 10
Current Milestone 1 Focus . . . 11
Current Situation . . . 11
1.2.2 Needs . . . 13
Purpose . . . 13
User Needs . . . 14
Needs of Sports Followers# . . . 14
Needs of Reporters, Organizers,# and transparent during these peak moments, instead Information Originators . . . 15
System Needs# . . . 15
Cross-Cutting Need: Dependability of Information# . . . 16
Data Reliability Needs# . . . 16
Assumptions and Plausibility . . . 16
Scope of This Section . . . 17
1.2.3 Ideas . . . 17
Purpose . . . 17
A Sports Information Platform for Local Puerto Rico Coverage . . . 17
Open Handling of Uncertainty . . . 17
Cross-Source Validation and Source Weighting# . . . 18
Source-Aware Presentation . . . 18
Data Freshness and Provenance . . . 18
Provisional# Corrections and Confidence Indicators . . . 19
Handling Uncertainty and Missing Data# . . . 19
Mobile-First Display . . . 19
Scope of This Section . . . 20
1.3 Scope, span, and synopsis . . . 20
1.3.1 Scope and Span . . . 20
Purpose . . . 20
In-Scope Coverage . . . 20
Local Puerto Rico Sports Leagues . . . 20
Reliability Modeling# Data Handling Under Uncertainty . . . 20
Mobile-First Platform Assumption . . . 21
Documentation-First Engineering# . . . 21

Out-of-Scope Elements . . . 21
Operational Constraints . . . 22
Informal Data Sources . . . 22
Limited Structure and Standardization . . . 22
Transparency Requirement . . . 22
Boundary Summary . . . 22
Synopsis . . . 23
Purpose . . . 23
Research Activities . . . 23
Documentation-as-Code . . . 24
Pull Request and Review Process . . . 24
CI/CD and Build Automation . . . 24
Task Tracking and Governance . . . 25
Team Coordination . . . 25
Milestone Planning and Retrospectives . . . 25
Conclusion . . . 26
Purpose . . . 26
Reliability Goals . . . 26
Transparency Goals . . . 27
Availability Goals . . . 27
Scope and Boundary Goals . . . 28
Trust and Governance Goals . . . 28
Measurability and Verification Considerations . . . 29
Conclusion . . . 29
Conclusion . . . 29
2 Descriptive part . . . 31
2.1 Domain description . . . 31
2.1.1 Domain Rough Sketch . . . 31
2.1.2 Terminology . . . 33
2.1.3 Domain Terminology in Relation to Domain Rough Sketch . . . 35
Purpose . . . 35
Mapping Terminology to Domain Themes . . . 36
From Broad Concerns to Fixed Terms . . . 36
Terminology Used for Information Status . . . 36
Terminology Used for Degree of Confidence . . . 36
Terminology Used for Source and Freshness Context . . . 37
Terminology Used for System Conditions . . . 37
Relation to the Rough Sketch . . . 38
2.1.4 Narrative . . . 38
Purpose . . . 38
Context: Local Sports Information Flow . . . 38

Actors and Typical Scenarios . . . 38
Narrative: What Happens Today (Problem Story) . . . 39
Narrative: Expected System Behavior (Tu Deporte Aquí) . . . 39
Why This Matters (Reliability + Transparency) . . . 39
2.1.5 Events, Actions, and Composite Behaviors . . . 39
Domain produced, compared, and updated. . . . 39
System Actions per Event . . . 40
Expected System Behaviors . . . 42
Notes for Future Testing Remarks . . . 44
2.1.6 #2.1.6 Function Signatures (Pseudocode at the moment, Not Event-Tied) . . . 44
Core Records (state holders) . . . 44
E1 Actions (Report Received) . . . 44
Fetching and Snapshot Storage . . . 45
Parsing and Normalization . . . 45
Canonical Merge (best-known current state) . . . 45
Conflict Handling . . . 46
E5 Actions (Data Becomes Stale) . . . 46
E6 Actions (Correction Published) . . . 47
User-Facing Queries . . . 47
Maintenance Note on Scope . . . 47
2.2 Requirements . . . 48
2.2.1 User Stories, Epics, Features . . . 48
Epic 1: Consolidated Puerto Rico Sports Information Feed . . . 48
Epic Epic 2: Conflict Detection Handling, Uncertainty, and Consistency Controls . . . 49
Epic 3: Graceful Degradation and User Trust Signals . . . 50
Epic 4: 3: High-Traffic Availability During Major Events . . . 50
Epic 5: 4: Reliability Testing Automation and Observability . . . 51
2.2.2 Personas . . . 52
Persona A: Sebastian Rivera, #The personas below ground the Superfan user stories in Section 2.2.1. They are meant . . . 52
Persona B: Coach Abiel Rullan, the Team Organizer . . . 53
Coach Abiel Rullan "The Team Organizer" . . . 53
Persona C: Andres Perez, final. . . . 54
Chris Ramos "The Platform Engineer Engineer" . . . 55
2.2.3 Domain Requirements . . . 56
Purpose . . . 56
Observed Domain Properties . . . 56
Domain Requirements . . . 57
2.2.4 Interface Requirements . . . 60
Purpose . . . 60
Shared Phenomena . . . 60

Interface Requirements . . . 60
2.2.5 Machine Requirements . . . 62
Purpose . . . 62
Notes on Current Status . . . 62
Machine Requirements . . . 63
Open Items . . . 65
2.3 Implementation . . . 66
2.3.1 Selected Implementation Fragments . . . 66
Purpose . . . 66
Scope and Assumptions . . . 66
Fragment 1: Representation of Data with Reliability Metadata . . . 66
Fragment 2: Conflict Detection and Resolution Strategy . . . 66
Fragment 3: Data Freshness and Staleness Indicators . . . 66
Fragment 4: User Interface Communication of Uncertainty . . . 67
Traceability to Requirements . . . 67
Notes . . . 67
3 Analytic part . . . 68
3.1 Concept Analysis . . . 68
Purpose . . . 68
Core Concepts . . . 68
Data Sources . . . 68
Reliability State . . . 68
Timestamp and Freshness . . . 68
Concept Relationships . . . 69
Reliability Interpretation . . . 69
User Trust Impact . . . 69
Conceptual Constraints . . . 69
Mobile-First Context . . . 69
Summary . . . 70
3.2 Validation and Verification . . . 70
Purpose . . . 70
Validation Approach . . . 70
Verification Approach . . . 71
3.2.1 Validation and Verification . . . 71
Purpose . . . 71
Validation Approach . . . 72
Verification Approach . . . 72
3.2.2 System Testing Perspectives . . . 73
Purpose . . . 73
Testing Approach . . . 73
4 Specific class topics . . . 75

4.1 Test-Driven Development (TDD) . . . 75
Purpose . . . 75
Role of TDD in the Project . . . 75
Application to Tu Deporte Aquí . . . 75
Example TDD Workflow . . . 76
Alignment with Milestone 1 . . . 76
TDD Walkthrough Example . . . 76
Conclusion . . . 78
Purpose . . . 78
From User Stories to Acceptance Criteria . . . 79
From Acceptance Criteria to Specification Examples . . . 79
From Acceptance Criteria to Specification Examples . . . 81
The Feature Files . . . 81
Example: A Real Scenario from `interface_requirements.feature` . . . 81
Traceability Matrix Mapping Between Scenarios and Tests . . . 82
Tool Usage . . . 84
Current Limitations . . . 85
Current Status and Next Steps . . . 86
Example: End-to-End Behavior Validation . . . 86
Conclusion . . . 86
4.3 Use of Debugging . . . 87
Purpose . . . 87
Role of Debugging . . . 87
Debugging Environment . . . 87
Expected Debugging Scenarios . . . 87
Scenario 1: Conflict Detection in Score Merging Display with Missing Data . . . 88
Scenario 2: Staleness Threshold Behavior . . . 88
Scenario 3: Source Outage and Degraded Mode Full Workflow Integration . . . 89
Integration with Testing . . . 89
Alignment with Project Goals . . . 90
Conclusion . . . 90
5 AI Disclosure . . . 91
Overview . . . 91
Tools Used . . . 91
Purpose of AI Assistance . . . 91
Human Ownership and Contribution . . . 91
Verification and Quality Control . . . 92
Academic Integrity Considerations . . . 92
Reflection on Learning . . . 92
Conclusion . . . 92

# 1 Informative part

## 1.1 Team

**Project Organization**

Tu Deporte Aquí is organized into five specialized teams to that support structured documentation, research, both documentation and feature preparation during Milestone 1.

Milestone 1 does not involve implementation or source code development. The across project milestones.

During Milestone 1, the focus is was on defining scope, system behavior, reliability expectations, documentation and transparency requirements system definition. In the current milestone, the project has transitioned into implementation, where teams are responsible for a mobile-first platform serving local Puerto Rico sports. building and integrating system components.

Each team is responsible for assigned a clearly defined specific domain area area. This allows work to ensure full coverage of system expectations before development begins. be distributed while still maintaining coordination across the system.

**Team Structure**

**Team 1 – Core Documentation & Project Standards**

Responsible for establishing the official documentation foundation:

- Expanding the A3 into a structured project overview
- Defining scope boundaries (local leagues only)
- Documenting assumptions (limited or no official APIs; reliance on informal sources) maintaining consistency and supporting integration across teams:
- Maintaining the main Updating README and project documentation index
- Defining shared glossary folder structure and documentation coding standards
- Configuring AsciiDoc build automation Documenting API endpoints created by feature teams
- Preparing the final compiled Milestone 1 Supporting deployment setup
- Applying professor feedback to documentation

This team does not implement features, but ensures documentation consistency, structure, that the work produced by other teams remains consistent and governance. properly documented.

**Team Members:**

- Alberto J. Rodríguez Nogueras (Lead)
- Gerald L. Rodríguez Rodríguez
- Jafet Hernández Arroyo

- Ángel L. Ramos Rodríguez
- Daniel C. Colón Rivera
- Alex M. Vázquez Galiano

#### Team 2 – Live Games Supabase Core & Match Coverage

Defines how live game information is displayed Platform

Responsible for backend foundation and trusted:

- What data is shown (scores, time, league, status) management:
- Handling delayed updates Setting up Supabase project
- Resolving conflicting score reports Designing database schema (users, games, standings, teams, athletes, news)
- Behavior when information is missing Inserting seed data
- Mobile-first UI messaging for provisional Implementing basic Row-Level Security
- Supporting other teams with data access

This team focuses on reliability under real-world uncertainty. acts as the central point for database-related decisions.

**Team Members:**

- Ángel Villegas (Lead)
- Luis X. Sorrentini Renta
- Noel A. Colón Mena
- Joaquín E. Arraiza Morales
- Ian G. Ramos Villafañe
- Cristian J. Massini Perez

#### Team 3 – Standings Live Games & Tournament Progress

Defines how standings are calculated Standings

Responsible for implementing game and communicated:

- Representation of local league standings views:
- Handling inconsistencies across informal sources Building /games and /standings pages
- Resolving or flagging conflicting standings Implementing API routes
- Fetching data from Supabase
- Displaying provisional or uncertain rankings scores, teams, and status
- Managing edge cases (postponements, incomplete data) Handling loading and error states

This team ensures structural integrity of league progression information. focuses on presenting

structured sports data through working UI pages.

**Team Members:**

- Naedra J. Feliciano (Lead)
- Natanael Massallo Soto
- Christopher J. Perez Rodríguez
- Yandiel Hernandez Otero
- Javier A. Ventosa Maldonado

#### Team 4 – Teams & Athletes News, Profiles & Alerts

Responsible

Defines expectations for profile information accuracy: content and profile-related pages:

- Required Building /news, /teams/[id], and optional profile fields /athletes/[id]
- Acceptable levels of staleness Implementing API routes
- Handling incomplete or outdated Displaying content and profile data
- Mobile-friendly indicators for uncertain information Handling loading and error states

This team focuses on data completeness and freshness constraints. making informational content accessible through the UI.

**Team Members:**

- Sebastian A. Santiago Perez (Lead)
- Edward J. Carde Velazquez
- Diego A. Hernandez Sanchez
- Pedro M. Cortes Ramos
- Edgar J. Gonzalez Vargas

#### Team 5 – News, Alerts Authentication & User System Transparency

Defines how system updates

Responsible for user authentication and uncertainty are communicated: session management:

- Criteria for publishing news vs urgent alerts Implementing login and signup
- Handling unverified or conflicting reports Managing user sessions
- Transparency indicators (last update time, confidence level) Protecting routes
- Messaging during degraded or uncertain states Handling authentication errors
- Displaying user state in UI

This team ensures user trust and system transparency. that user-specific functionality works

correctly.

**Team Members:**

- Juan R. Rivera Rodríguez (Lead)
- Diego A. Ríos Morales
- Lianette Alberto Muñoz
- Marcos F. Arraiza Aviles
- Ojiani S. Figueroa Cruz

#### UI Strike Team

A cross-team group responsible for shared UI components and layout consistency.

Responsibilities include:

- Building homepage
- Creating shared layout components (navbar, layout wrapper, navigation)
- Ensuring consistent styling across pages
- Supporting mobile-first design

This group coordinates UI decisions across feature teams.

### Roles and Responsibilities

#### Developers

Developers are responsible for:

- Creating and completing GitHub Completing assigned issues
- Conducting research aligned with team goals Implementing features or documentation updates
- Writing structured code or documentation depending on their team
- Following the Issue Creation Guidelines
- Submitting work through pull requests
- Ensuring acceptance criteria are satisfied

All work must be issue-based and documented. tracked through GitHub issues.

#### Team Leads

Team Leads are responsible for:

- Coordinating work within their assigned team
- Conducting first-level review of completed issues Reviewing work before it is merged
- Ensuring alignment with Milestone 1 objectives

- Tracking participation and progress

Team Leads help maintain quality and consistency before escalation tasks are completed according to managers.

**Managers:** requirements - Communicating with other teams when needed

This role helps reduce integration issues between teams.

#### Managers

Managers:

- Anthony Martínez
- Fernando Torres

Managers are responsible for:

- Reviewing issue clarity issues and completeness pull requests
- Assigning urgency and difficulty levels
- Approving pull requests after review
- Closing issues upon satisfactory completion
- Ensuring milestone alignment and fairness progress
- Resolving cross-team issues

Managers ensure governance and structured progression of that the project. project stays aligned with milestone goals.

### Workflow and Governance

The project follows an issue-driven and PR-only workflow with branch protection enabled.

All work must begin with a GitHub Issue created using the standardized Issue Creation Guidelines. Issues must:

- Follow the approved title format
- Use the required content template
- Include clearly defined acceptance criteria
- Specify urgency and difficulty

Issues may involve documentation, research, feature definition, lecture topic tasks, or coordination work.

For documentation scaffold sections specifically, issues must reference the master documentation scaffold issue (#53) to ensure traceability within the single compiled document structure. pull request review.

Workflow:

1. Issue is created following the standardized template and assigned
2. Developer creates works on a feature branch or update
3. Developer modifies only the assigned scope 4. Pull Changes are submitted through a pull request references 4. CI runs to validate the corresponding issue project
4. CI workflow builds documentation artifacts automatically 6. Team Lead performs review 7. reviews 6. Manager approves and merges

No direct pushes to protected branches This process ensures that changes are permitted for non-admin users. reviewed and integrated consistently.

**Current Milestone 1 Focus**

Milestone 1 emphasizes: The project is currently focused on implementation.

This includes:

- Clear scope definition (local Puerto Rico sports only) Building UI pages using Next.js
- Explicit documentation of uncertainty and failure scenarios Implementing API routes
- Mobile-first user interface expectations Integrating Supabase
- Transparency in data reliability Connecting frontend and freshness backend
- Structured documentation before implementation

This structured Implementing authentication

The team model ensures structure supports this by distributing responsibility across features while maintaining coordination through shared ownership, accountability, and disciplined engineering practices prior to any system development. workflows. === 1.2 Current situation, needs, ideas ==== 1.2.1

**Current Situation**

This section describes the current state of how sports information availability in about local Puerto Rico prior to the implementation of Tu Deporte Aquí.

**Overview**

Sports events is currently produced, shared, and consulted, without assuming any particular system.

Local sports information in Puerto Rico is distributed not provided through a single coordinated source. Instead, it appears across multiple independent platforms, including channels. In practice, people rely on a combination of league websites, team pages, social media posts, sports media, news outlets, third-party platforms, and informal community reporting channels. There to follow schedules, scores, standings, and event updates.

This description is no centralized or standardized system for accessing this data. based on project research into how local leagues publish updates, how people discover events, and how standings, scores, and live data are communicated.

#### Data Distribution Environment

Information is typically published through: - League Sources in the Current Environment

Sports information appears through several types of sources:

- official league websites
  - Social * team and league social media pages
- sports media posts (e.g., Twitter, Instagram)
  - News outlets
  - Independent * independent reporters and community pages
- third-party sports platforms

These sources often operate independently, with no synchronization or standardization.

#### Key Limitations

The current environment presents several limitations: - Data fragmentation across multiple platforms - Lack differ in format, timing, and level of official APIs detail. Some provide structured data, while others provide short updates intended for many leagues - Inconsistent update frequencies - Limited standardization in reporting formats immediate awareness.

#### Reliability Challenges

Sports data in Puerto Rico Fragmentation of Information

Information about the same event is frequently: - Delayed due often split across multiple sources.

For example:

- a league page may publish an official result after a reporter at the venue
- a team account may post a score without additional context
- a media outlet may summarize standings without showing the full table

To understand a single event, a person may need to consult several sources.

#### Irregular Timing and Delay

Updates are not synchronized across sources.

Observed patterns include:

manual posting of scores * gaps between updates during active games * delayed publication of final results or slow reporting - Conflicting when multiple standings sources provide reflecting different values - Incomplete when certain fields (score, time, status) are missing - Unverified when information comes from informal points in time

Two sources checked at the same moment may show different versions of the same event.

#### Inconsistency and Incompleteness

Information may differ in both content and completeness.

Examples include:

- different scores reported for the same game
- inconsistent standings or rankings
- missing time indicators or status labels
- updates that do not indicate whether they

These issues are more noticeable during high-demand events such as playoffs. live or final

These differences often result from manual entry, different publication practices, and lack of coordination between sources.

#### Impact on Users

Users face several challenges: - Difficulty determining Consequences for People in the Domain

The current situation affects different roles:

- fans and families may have difficulty deciding which information is correct
  - Need to cross-check multiple trust
- players and coaches may need to verify schedules and results across sources
  - Confusion during conflicting * leagues and organizers may see outdated or delayed updates
  - Reduced trust in available information

In many cases, incorrect partial information causes more harm to user trust circulate * reporters and community contributors often publish quickly without full confirmation

Following local sports often requires comparing multiple sources rather than clearly communicated missing information.

This situation highlights the need for relying on a system that prioritizes transparency and reliability when handling uncertain data. single consistent view.

### 1.2.2 Needs

#### Purpose

This section explains the main needs that justify building Tu Deporte Aquí. These needs come identified from the constraints, assumptions, and real-world conditions identified throughout Milestone 1. current situation in Section 1.2.1. The needs are organized into three categories: user needs, focus is on what people involved in local Puerto Rico sports need, not yet on how a system needs, and data reliability would address those needs.

**User Needs**

In Puerto Rico, Stakeholder Context

The current situation shows that sports fans, coaches, reporters, and community members depend on information in Puerto Rico is spread across league websites, social media, media posts, news outlets, and other informal reporting channels. Those sources to follow local sports. Because this information is spread across many platforms, several clear needs arise:

**Centralized access to local sports information**

Users need a single, mobile-first platform that brings together game scores, standings, schedules, and athlete updates from local Puerto Rico leagues. Currently, staying informed usually means checking multiple sources that update at different times—or sometimes not at all.

**Clarity during uncertainty**

When information is times, use different formats, and may be delayed, incomplete, or conflicting, users need the platform in conflict with one another.

Under those conditions, several groups are affected. Sports followers want to clearly indicate know what is confirmed, what happening now and whether the information they found is still uncertain, dependable. Reporters, organizers, and what other information originators want updates and corrections to reach the public without losing their source or context. These needs arise before any decision is missing. For example, during a live BSN playoff game, users made about what kind of system should be able built.

## Needs of Sports Followers#

**Accessible local sports information**

People who follow local sports need to distinguish between a confirmed final score find current scores, schedules, standings, and a provisional update taken related updates without searching across many unrelated channels. This need follows from social media.

Trustworthy the fragmented information environment described in Section 1.2.1.

**Timely awareness of what is happening**

Followers need information early enough to remain useful while events are still relevant. When updates arrive too late, people can no longer follow a game as it unfolds or react to changes that matter to them.

**Clarity when information is incomplete or disputed**

Followers need to understand when available information is missing details, when reports disagree, and when a situation is still unresolved. Without that clarity, they are left to guess which version is most believable.

**Confidence during high-demand high-interest moments**

Playoffs, During playoffs, tournaments, and breaking news events are when users rely other heavily followed events, followers depend more on the platform the most—and when data sources sports information and are affected more likely by delays or contradictions. At those moments, they need to lag know whether the available information is stable or disagree. The

platform must remain accessible still subject to change.

## Needs of Reporters, Organizers,# and transparent during these peak moments, instead Information Originators

**Faithful transmission of failing silently updates**

People who publish or presenting unverified relay sports information as final.

**Source attribution and freshness indicators**

Users need visibility into their updates to remain connected to what they actually reported. If scores, schedules, or status changes are repeated without context, the public can no longer tell where each update comes the information came from and when or whether it was last refreshed. Without this context, it is difficult to determine whether a score reflects a live update, a delayed post, altered later.

**Visible corrections and revisions**

When reporters, leagues, or outdated cached data.

**System Needs#**

Tu Deporte Aquí operates organizers correct previously shared information, they need those corrections to remain visible as corrections rather than disappear through silent replacement. This matters in an environment where official APIs are limited or unavailable, and many data sources are informal. The system must be designed around this reality:

**Conflict detection and resolution**

The system needs historical changes may occur without a way to detect when different sources report conflicting values for formal correction workflow.

**Recognition of differing source authority**

Not all originators play the same data point (such as different scores for a single game) and apply clear policies to resolve or flag those conflicts. Disagreement between sources should be treated as an expected condition, not a rare exception.

**Graceful degradation under partial failure**

If one or more data sources become unavailable, delayed, or unreliable, role in the system should continue serving previously validated sports information instead of showing errors or empty states. Fallback behavior should be consistent and predictable.

**Scalability during peak traffic**

The system must handle traffic spikes during major events like playoffs and championship games, when demand is highest ecosystem. Official league channels, institutional pages, journalists, and reliability is critical. Core information—scores, standings, informal observers contribute differently, so those differences in authority and game status—should responsibility need to remain prioritized under heavy load.

**Standardized transparency messaging**

The system needs a consistent way visible.

## Cross-Cutting Need: Dependability of Information#

Across all stakeholder groups, people need to communicate degraded states, stale data, and uncertainty. Error messages, loading indicators, and missing-data notices should be clear and uniform across judge how dependable sports information is.

In the platform.

### Data Reliability Needs#

Since Tu Deporte Aquí aggregates data from current situation, dependability is difficult to assess because sources that vary in structure, timeliness, are fragmented, update frequencies are inconsistent, and accuracy, reliability is some reports are delayed, incomplete, or contradictory. People therefore need enough context to interpret the status of the information they encounter.

That context includes:

- where a central concern:

  **Provenance and traceability**

  Every piece of displayed information must have a traceable origin. The system should record which source provided each update, came from - when it was received, and last known to be updated - whether it appears settled, still provisional, or openly disputed

- whether a later correction has been modified since ingestion. This supports user trust and simplifies internal debugging.

Handling of changed the earlier picture

**Source attribution, freshness, provisional status, and unverified data**

The system must clearly differentiate between confirmed information and data that is still preliminary or unverified. Provisional updates should be labeled appropriately rather than presented with traceability are treated here as parts of the same level need: helping people judge the dependability of confidence as confirmed results.

**Staleness awareness**

Standings, profiles, and schedules become outdated at different rates. The system needs defined thresholds for acceptable staleness and a mechanism to flag or hide the information that exceeds those limits.

**Correction and revision tracking**

Local leagues may revise scores, standings, or schedules after initial publication, sometimes without formal notice. The system should support basic revision tracking so corrections are recorded and previous values remain available for audit purposes. to them.

### Assumptions and Plausibility

The following assumptions support the needs described are used in this section: section. They are plausible because they match the conditions already documented in Section 1.2.1 and the research collected during Milestone 1:

- Most local Local Puerto Rico leagues do not provide sports information is often encountered through a mix of official APIs or machine-readable data feeds.
- Data sources include league websites, university athletic department pages, social media posts, and informal reporting channels. unofficial channels rather than through one uniform publication mechanism.
- Reporting formats, update frequency, The timing, structure, and completeness of updates vary significantly across noticeably between leagues and sources.
- Historical corrections may occur without formal announcements.
- The platform is mobile-first Corrections and web-based; native mobile applications late revisions are not included in Phase 1. possible, especially when reporting depends on manual publication or informal communication channels.
- No implementation Different stakeholder groups share an interest in current sports information, but they do not all approach it from the same role, level of authority, or source code is produced during Milestone 1; this section documents needs based only on research and analysis. immediate goal.

**Scope of This Section**

This section defines *what* is needed and *why*, without going into implementation details. Technical solutions, architectural decisions, the needs that arise from the current situation and feature-level specifications are addressed explains why those needs matter. It does not choose a solution, define features, or state requirements. High-level ideas for addressing these needs appear in Section 1.2.3, while requirements are specified later sections of the documentation. in Section 2.

## 1.2.3 Ideas

**Purpose**

This section explains outlines the main ideas that guide for Tu Deporte Aquí. These ideas build on respond to the conditions and needs identified described in Milestone 1 Section 1.2.2 and describe give a high-level view of the direction kind of system the team intends to build.

**A Sports Information Platform for Local Puerto Rico Coverage**

One central idea is to build a platform focused on local Puerto Rico sports. The platform is meant to bring together scores, schedules, standings, and related updates that are currently spread across league websites, social media posts, news outlets, and other reporting channels.

This idea follows from the fragmented situation described earlier. Rather than expecting followers to check many sources on their own, the system-to-be would gather relevant information into one place.

**Open Handling of Uncertainty**

Another main idea is that the platform should take, without defining specific technical solutions.

### Cross-Source Validation and Source Weighting#

Tu Deporte Aquí collects data from multiple informal sources, where disagreements are common.

#### Comparison before display

When different sources report different values for the same data point, treat uncertainty as normal rather than exceptional. Since local sports information may be delayed, incomplete, or contradictory, the system platform should compare them before showing not present all data as equally settled.

At a result. If values match, confidence increases. If they conflict, high level, this means the system system-to-be should flag the issue separate information that appears stable from information that is still uncertain, and it should make conflicts or missing information visible instead of silently choosing one.

#### Source trust levels

Not all sources are equally reliable. hiding them.

#### Source-Aware Presentation

The platform should assign trust levels to different present sports information together with enough source types—for example, an official league website should carry more weight than context for people to judge it. The main idea is not only to show a social media post. This weighting should be intentional score or status, but also to show where that information came from and documented.

#### Conflict visibility

Conflicts should be logged internally how recent it is.

This idea follows from the need for debugging. When relevant, they dependability described in Section 1.2.2. If information comes from sources with different levels of authority or different update timing, the platform should also be shown to users through clear indicators, preserve that context instead of presenting all updates in the data as if it were fully certain. same way.

### Data Freshness and Provenance

Users need to understand how recent the information is and where it came from.

#### Timestamps and source labels

Key data elements scores, standings, profiles, schedules should display when they were last updated and which source provided them (for example, "via BSN official site"). This allows users to judge reliability on their own.

#### Update frequency expectations

The platform should communicate realistic refresh rates Support for each data type. Live scores may have small delays, while standings may update only after games are finalized. Making this clear helps reduce confusion.

### Provisional# Corrections and Confidence Indicators

Since much of the data comes from informal channels, the system must clearly separate confirmed information from unverified data.

**Provisional labeling**

Information that has not been confirmed by a trusted source such as early live scores from social media—should be clearly marked as provisional.

**Confidence progression**

Change Over Time

The platform should use simple labels ("Confirmed," "Provisional," "Unverified") or visual cues (badges, muted styles) to show reliability. As more sources confirm a value, allow for the confidence level should update visibly so users fact that sports information can see how the data stabilizes over time.

### Handling Uncertainty and Missing Data#

Uncertainty change after it is part of the platform's environment and should first reported. Scores may be handled openly.

**Standardized messaging**

Delayed scores, incomplete standings, and outdated profiles should follow a consistent visual corrected, schedules may shift, and textual format so users can quickly understand what previously incomplete information may later be filled in.

The idea here is happening.

**Degraded and no-data states**

If that the system has limited data, it system-to-be should show not assume that the last known first reported value with an "as of" indicator. If information is completely unavailable, always the platform final one. It should display a clear "not available" state instead of leaving empty spaces or unclear placeholders. make room for updates, corrections, and later clarification.

### Mobile-First Display

As a mobile-first web platform, all of Access

Another idea is to treat the ideas above must work within small screen constraints.

**Information hierarchy**

Essential details score, game status, league, time should appear first. Supporting details such platform as source labels, timestamps, and confidence indicators mobile-first. Many followers check sports information from their phones during games or while moving between activities, so the system should appear be designed with small-screen access in secondary or expandable views.

**Compact indicators**

Icons, color coding, and short labels are preferred over long explanations. The same visual

language for freshness, confidence, and provisional states mind.

At a high level, this means the most important information should be used consistently across live scores, standings, and profiles easy to reduce cognitive load. reach quickly, without depending on large layouts or long explanations.

**Scope of This Section**

This section defines the conceptual direction gives a high-level view of how the platform. team intends to address the needs from Section 1.2.2. It states the main ideas behind the system-to-be, but it does not include define detailed behavior, technical implementation mechanisms, or architectural details, which are covered later in the documentation. formal requirements.

# 1.3 Scope, span, and synopsis

## 1.3.1 Scope and Span

**Purpose**

This section defines the boundaries, coverage limits, and operational constraints of Tu Deporte Aquí during Milestone 1. Aquí. Clearly defining scope prevents scope creep, ensures alignment across teams, and establishes realistic expectations before implementation begins.

Milestone 1 during implementation.

The project has transitioned from documentation and system definition into implementation. The scope now focuses exclusively on documentation, research, and structured building a working system definition. No implementation or source code development is included in this milestone. while maintaining the constraints previously defined.

**In-Scope Coverage**

The system scope for Milestone 1 includes:

**Local Puerto Rico Sports Leagues**

- Coverage is limited to local leagues within Puerto Rico.
- National and international leagues are excluded.
- Emphasis is placed remains on leagues where official APIs may not exist.

This focus ensures realistic alignment with data constraint reflects the availability and milestone constraints. reliability of data sources.

**Reliability Modeling# Data Handling Under Uncertainty**

The system will define expected behavior under: must handle:

- Delayed updates
- Conflicting reports

- Missing information
- Corrected or revised results

Explicit documentation of uncertainty handling is part of the defined scope. These cases are now considered during implementation, not only as documented scenarios.

**Mobile-First Platform Assumption**

All feature definitions and UI expectations assume

The system is implemented as a mobile-first, web-based experience. mobile-first web platform.

This includes:

- Compact information display
- Clear status indicators
- Prioritization of essential game data
- Responsive layout across devices

**Documentation-First Engineering#**

Milestone 1 Core Implementation Scope

The current milestone includes:

- Structured documentation scaffold UI implementation using Next.js
- Defined system behavior and assumptions Backend API routes
- Research-based reliability strategies Integration with Supabase
- Governance Authentication and workflow protected routes
- Display of real data in the frontend

This defines the transition from system definition to a working application.

No backend, frontend, or database implementation is included at this stage.

**Out-of-Scope Elements**

The following elements are explicitly excluded from Milestone 1:

- Implementation remain out of application code scope:
- Integration with official league APIs Real-time data updates (e.g., polling or WebSockets)
- Automated web data scraping mechanisms
- National or international league coverage
- Real-time Advanced performance optimization
- Production deployment Complex user roles or hosting infrastructure admin systems

These exclusions ensure that Milestone 1 remains focused on limit system definition rather than execution. complexity and keep the focus on core functionality.

#### Operational Constraints

Tu Deporte Aquí operates under the following constraints:

#### Informal Data Sources

- Data may originate from websites, reporters, or social media.
- Sources may conflict or provide delayed updates.
- Official verification mechanisms may not exist.

#### Limited Structure and Standardization

- Local leagues may lack standardized reporting formats.
- Inconsistent update timing must be expected.
- Historical corrections may occur without formal notice.

#### Transparency Requirement

Because of data uncertainty, the system must:

- Communicate provisional states
- Display last update times
- Indicate possible staleness
- Support traceability of changes

These constraints directly influence system design decisions affect how data is presented and interpreted in later phases. the system.

### Boundary Summary

Tu Deporte Aquí, during Milestone 1, Aquí is a documentation-driven project system focused on:

- Local Puerto Rico sports
- Reliability modeling
- Transparency definition Handling unreliable or inconsistent data
- Scope discipline Providing transparent information to users
- Structured governance

By clearly defining these boundaries, the project ensures disciplined progress Maintaining clear scope boundaries during implementation

These boundaries guide development decisions and alignment help ensure consistent progress across all teams prior to implementation. teams. ==== 1.3.2

**Synopsis**

Tu Deporte Aquí is a mobile-first, web-based platform designed to centralize addresses the problem of following local sports information for local in Puerto Rico leagues and collegiate teams.

Currently, sports data is scattered when updates are spread across multiple sources such as websites and may differ in timing, detail, and reliability.

Scores, standings, schedules, and event updates are commonly published through league pages, social media accounts, sports media, often resulting in delayed, incomplete, or conflicting information. This makes reporters, and third-party platforms. Because of this, it can be difficult for users to access reliable determine what information is current, what is uncertain, and consistent updates. how much confidence can be placed in a given update.

The platform project focuses on addressing these challenges by prioritizing reliability and transparency. Instead of hiding uncertainty, the system communicates verification status, confidence levels, and update timestamps to help users understand the quality of the information presented.

Phase 1 of the project is limited to local Puerto Rico sports coverage. sports. National and international leagues are explicitly out of coverage is outside its scope.

Tu Deporte Aquí aims to provide a clear, trustworthy, The system does not assume that all available information is complete or consistent. Updates may be delayed, partial, conflicting, or later corrected. For that reason, the project places attention on how information is presented, including its state, timing, and accessible way for users level of confirmation.

The goal is to follow improve how local sports while acknowledging the real-world limitations of the available data. information is organized and interpreted in situations where sources differ and conditions are imperfect. === 1.4 Other Activities than Development

**Purpose**

Milestone 1 of Tu Deporte Aquí emphasizes documentation, research, and structured system preparation rather than implementation. This section describes the non-development activities that contribute to system reliability, transparency, and organized collaboration.

These activities ensure that support the project evolves through disciplined engineering practices before any source code is written.

**Research Activities**

Research plays a central role in implementation of Tu Deporte Aquí.

While Milestone 1. Key 1 focused on research efforts include:

- Investigating reliability strategies for conflicting and delayed sports data
- Defining scope boundaries documentation, the current milestone emphasizes implementation. However, supporting activities such as documentation, coordination, and out-of-scope conditions

- Exploring data lineage workflow management remain necessary to ensure consistency and audit trail strategies
- Studying transparency indicators for uncertain information
- Analyzing requirements classification (functional vs non-functional) integration across teams.

These research tasks ensure that system behavior is well-defined under real-world constraints. activities help maintain structure and reduce errors during development.

**Documentation-as-Code**

The project adopts continues to use a documentation-as-code model using AsciiDoc and GitHub.

This includes:

- Structured folder hierarchy under /docs
- Modular section files linked through include:: directives
- Version-controlled documentation
- Pull-request-based content integration updates
- Automated HTML/PDF build via builds through CI workflow

This approach ensures traceability, consistency,

Documentation is updated alongside implementation to reflect system behavior and reproducibility of documentation. design decisions.

**Pull Request and Review Process**

All contributions are integrated through a PR-only PR-based workflow.

The process includes:

1. Issue creation using the standardized template
2. Feature branch creation
3. Focused modification of assigned section Implementation or documentation update
4. Pull request submission
5. Automated CI build verification
6. Team Lead review
7. Manager approval and merge

This structured process ensures quality control that changes are reviewed before being integrated and accountability. reduces inconsistencies between teams.

**CI/CD and Build Automation**

The project includes uses a GitHub Actions workflow that:

- Compiles Builds the master AsciiDoc document project and documentation
- Verifies include structure
- Generates HTML and PDF artifacts formatting
- Detects build or formatting errors before merging

This automation helps prevent broken builds and ensures documentation integrity before approval. that changes do not introduce issues.

#### Task Tracking and Governance

All work is tracked through GitHub Issues.

Each issue includes:

- Clear Defined objective
- Defined acceptance Acceptance criteria
- Urgency and difficulty rating
- Assigned contributor
- Milestone linkage

Documentation scaffold sections association

Documentation-related issues reference the master scaffold issue (53) to maintain [.removed]#structural traceability. structure.

This governance structure prevents duplication and supports milestone compliance. allows work to be distributed while still keeping visibility across teams.

#### Team Coordination

Collaboration is supported through:

- Team-based specialization (five domain-focused teams)
- Regular coordination discussions responsibilities
- Milestone planning Cross-team communication when dependencies exist
- Alignment on scope Coordination between frontend, backend, and assumptions data teams
- Shared responsibility for reliability modeling

These coordination practices ensure consistent interpretation use of requirements across teams. components and APIs

Coordination is necessary because features depend on shared data and structure.

#### Milestone Planning and Retrospectives

Milestone 1 Iteration

The project follows milestone-based development.

This includes:

- Clear completion criteria - Defined deliverables (compiled documentation)
- Explicit failure and uncertainty modeling implementation goals
- Iterative refinement through updates based on feedback Retrospective reflection ensures continuous improvement in - Adjustments to documentation quality and team workflow. structure as the system evolves

Unlike Milestone 1, changes now reflect actual system behavior rather than planned behavior.

**Conclusion**

Tu Deporte Aquí extends includes activities beyond software implementation. Research, writing code. Documentation, coordination, and structured documentation, governance modeling, workflows support implementation and automated validation form the foundation of the project. reduce integration issues.

These non-development activities help ensure that when implementation begins, it the system is grounded in clearly defined scope, reliability expectations, built consistently and transparent engineering practices. that changes remain traceable across milestones. === 1.5 Derived Goals

**Purpose**

The following goals are derived from the identified current situation, system identified needs, and proposed ideas for Tu Deporte Aquí. These goals translate contextual analysis into structured and actionable objectives that guide documentation, reliability modeling, and future implementation.

These goals serve as a bridge between represent intended directions for the Informative Part system rather than strict requirements.

They help guide design decisions and ensure that implementation remains aligned with the Descriptive and Analytic sections realities of the document. local sports data.

**Reliability Goals**

**G1 – Conflict-Aware Data Handling**

The system shall explicitly detect and manage is expected to handle conflicting sports data received from multiple sources.

This includes: involves:

- Identifying Detecting inconsistencies across sources
- Flagging Distinguishing between confirmed and provisional values
- Preserving traceability of overrides Allowing updates to replace earlier information when needed

This goal ensures reliability under reflects the nature of informal and semi-structured data conditions. sources where discrepancies are common.

#### G2 – Correction Transparency

The system shall support traceability of corrections should make it possible to scores, standings, understand how and game statuses. why data changes over time.

This includes:

- Tracking Identifying the source attribution of updates
- Recording update timestamps when changes occur
- Maintaining lightweight audit a simple history for critical of important fields

This goal enables reconstruction of data changes and improves accountability. supports traceability without requiring complex tracking systems.

### Transparency Goals

#### G3 – User-Facing Uncertainty Indicators

The system shall clearly should communicate when information is provisional, stale, incomplete, or unverified. may not be fully reliable.

This includes:

- Displaying Showing last update time times
- Showing Indicating when data confidence indicators may be outdated or provisional
- Providing context for conflicting information when sources conflict

This goal strengthens user trust and informed interpretation helps users interpret the information instead of data. assuming it is always correct.

#### G4 – Explicit Assumption **Documentation** Awareness

The system shall clearly document operational assumptions, including: is expected to operate under known limitations, such as:

- Limited or no Lack of official APIs
- Reliance on websites and social media informal sources
- Potential delays Delays in live updates

This goal prevents Making these assumptions visible helps avoid unrealistic expectations and scope ambiguity. expectations.

### Availability Goals

#### G5 – Graceful Degradation

The system shall remain operational should continue to function even when one or more data sources are unavailable. is incomplete or delayed.

This includes:

- Serving cached or Displaying previously validated available data when updates fail
- Avoiding complete full system failure due to partial outages missing information
- Providing transparency Informing users when updates are delayed data may not be current

This goal ensures resilience in a low-structure data environment. prioritizes usability over completeness.

### Scope and Boundary Goals

#### G6 – Local League Focus

The system shall is intended to focus exclusively on local Puerto Rico sports leagues during initial development phases.

National and international leagues are explicitly out of scope. leagues.

This goal prevents keeps the scope creep manageable and ensures realistic milestone alignment. aligned with available data.

#### G7 – Mobile-First Design Alignment

All documentation and feature definitions shall assume The system is expected to prioritize a mobile-first user experience.

This includes:

- Compact UI messaging
- Clear visual indicators
- Prioritized Presenting essential information clearly
- Keeping layouts simple and readable
- Ensuring

This goal ensures usability consistency across teams. different screen sizes

This reflects how users are likely to access the platform.

### Trust and Governance Goals

#### G8 – Structured Documentation Before Implementation Development Approach

The project shall fully define system behavior, constraints, aims to maintain clear structure in both documentation and uncertainty scenarios before any code is written. implementation.

This includes:

- Explicit failure scenario modeling Keeping system behavior well-defined
- Defined Maintaining traceability strategies of changes
- Clearly structured Updating documentation scaffold alongside development

This goal aligns with Milestone 1's emphasis on documentation-first engineering. supports consistency as the system evolves.

#### G9 – PR-Driven Accountability

All contributions shall be Collaboration

The team is expected to continue using issue-based work and integrated through pull request review with CI validation. review.

This ensures: helps:

- Traceability of Track changes over time
- Quality control Ensure work is reviewed before merging
- Structured collaboration Maintain coordination across teams

### Measurability and Verification Considerations

These goals are not strict requirements but reference points that guide development decisions.

As implementation progresses, some of these goals may be refined or translated into more specific requirements.

### Conclusion

These derived goals are conceptually verifiable through:

- Documentation completeness review
- Explicit modeling of failure scenarios
- Inclusion of uncertainty indicators in design descriptions
- Defined traceability metadata models
- Milestone alignment checks

Future implementation phases will operationalize these goals through structured requirements and validation strategies.

### Conclusion

The derived goals formalize provide direction for Tu Deporte Aquí's commitment to reliability, transparency, scope discipline, and structured engineering practices. Aquí while allowing flexibility during implementation. They ensure that system design decisions remain aligned with real-world reflect the constraints of local sports data and the Milestone 1 requirement of documentation-first

development. help keep development aligned with realistic expectations.

# 2 Descriptive part

## 2.1 Domain description

### 2.1.1 Domain Rough Sketch

While shaping the idea for **Tu Deporte Aquí**, we kept running into This section keeps# the same real-world constraint: sports information early discussion in Puerto Rico a rough form. It does not live in one try to clean database. It everything up yet. The point is scattered across league sites, news outlets, social media, and informal sources, and those channels often disagree or update at different speeds. Because of that, the A3 treats reliability as the main lens for to leave the domain, not just “having features.” The platform has original concerns visible so later sections can trace their concepts back to consistently show accurate standings, results, and athlete updates, while staying understandable when [.inserted]#them.

#A lot of the underlying inputs are messy.

**Distributed sources and conflicting information**

A central theme is that early conversation kept circling around the platform must ingest data from multiple leagues and informal reporting channels, which naturally increases same frustration: people want to follow local sports in Puerto Rico, but the chance of incorrect or outdated information appearing. The domain therefore includes disagreement as a normal scenario, where two sources may report conflicting information and the system must still behave predictably.

**Delays and missing updates**

We also focused on the fact that updates can arrive late or does not come from one place, does not arrive at all, especially for local events. The platform must handle incomplete, delayed, or missing inputs without turning that uncertainty into user confusion.

**High demand moments**

Another part one speed, and does not always match across sources.#

Some# of the domain statements and situations that came up were close to the [.inserted]#following:

* "I saw one score on Instagram and a different one in a group chat." * "Sometimes the reporter posts first, but later the league page says# something else." * "A game can look live in one place and finished in another." * "During playoffs everybody starts checking at once, and that is exactly when pages get slow or updates stop." * "If nothing new has been posted, I still want to know what the last known score was and when it was last changed." * "A blank space is worse than saying that reliability the information is missing." * "If a result gets stress tested when corrected later, people care start arguing about which version was the most, such as playoffs, major tournaments, real one." * "Not every local league posts with the same format, and breaking news. These moments create high traffic pressure, some barely post at all." * "Sometimes you only get a screenshot, a short post, or a message from someone who was there." * "When information changes, I want to know whether it was corrected, [.inserted]#confirmed, or just replaced."

Another thing that kept coming up was that people were not only asking for# scores. They were also trying to figure out whether the information in front [.inserted]#of them deserved trust. In practice, that meant questions like these:

* "Who posted this first?" * "Was this posted by the league, a reporter, or just a community page?" * "How old is this update?" * "Is this final, still developing, or just the only thing anyone has posted# so availability [.inserted]#far?"

There were also rough discussion points that were not settled yet# and performance under load are part of were [.inserted]#left open on purpose:

#* If an official source posts late and a reporter posts earlier, which one should people follow in the core domain expectations.

Clear behavior meantime? [.inserted]* If two sources disagree, should one be shown, should both be shown, or should the result be held back? [.inserted]* How old can a live update be before people stop treating it as live? [.inserted]* What is the minimum amount of information needed before a game should be shown as in progress? [.inserted]* If all sources go quiet during degraded states:: We identified a game, should people see the last known value, a warning, or both?#

#A recurring tension in these discussions was that failures are failure is not only technical events, they are user-facing experiences. here. If fallback strategies and error handling are weak, failures surface directly to users. In this domain, the platform needs to communicate uncertainty and degraded states clearly, and still keep user messages informative even when dependencies fail a source is late, disappears, or disagrees with another source, people experience that as confusion right away. The problem is not just "the data is unreliable.

**Trust as the main outcome**

> All of these concerns tie back messy." The problem is that fans, coaches, reporters, and organizers still need to make sense of what is happening while the end result the A3 prioritizes: long term user trust. When information is wrong, late, incomplete, delayed, or unclear during important events, users lose confidence quickly contradictory.#

#Another recurring point was that the hardest moments are usually the ones people care about most. Playoffs, tournaments, rivalry games, and late-game updates were brought up more than once because those are the platform becomes perceived as unreliable over time.

Overall, moments when more people are checking, more posts are appearing, and mistakes become visible fast.#

At this stage,# the domain rough sketch describes is not meant to explain the domain neatly. It is a reliability driven platform that remains available during peak demand, stays consistent when sources conflict or arrive late, record of the situations, phrases, and communicates reliability status unresolved questions that kept appearing in a way users early discussion. Later sections can understand, supported by testing that targets stress organize these into terminology, actions, requirements, and failure conditions rather than only normal operation. [.inserted]#validation steps.

### 2.1.2 Terminology

#### Verification Status Terms

This section defines the official terminology used in Tu Deporte Aquí. These terms must be used consistently across documentation and future testing scenarios, especially when describing reliability, transparency, throughout the document to maintain consistent meaning.

#### Core Domain Terms

**Sporting Event**

A game, match, or competition that people may follow.

**Source**

An origin of information about a sporting event. Examples include league websites, team pages, sports media, reporters, and uncertain third-party platforms.

**Source Report**

A specific piece of information states.

**Confirmed**

Information validated produced by an official a source, such as a score update, status, or highly trusted source and considered accurate at the time correction.

**Event Record**

The maintained representation of display.

**Unverified**

Information what is currently known about a sporting event.

**Standings Snapshot**

A reported by ordering or ranking of teams at a single source specific time.

**Correction**

A change to previously reported information based on new or informal updated data.

#### Dependability and Verification Terms

**Source Trust Level**

A general assessment of how dependable a source without tends to be.

**Verification State**

The level of confirmation from an of a reported value.

Defined states:

- **Unverified** — not yet confirmed
- **Confirmed** — supported by corroborating or authoritative provider.

**Developing**

Time-sensitive information that is still being verified and may change as more sources publish updates.

**Conflicting**

A state where multiple sources report different values for the same event or field (e.g., score, status).

**Delayed**

Information submitted significantly after the event occurred or updated slower than

- **Final** — not expected due to source latency.

**Corrected**

change under normal conditions

**Provisional**

Information that was previously published is available but later changed due to an official correction or reconciliation may still change.

NOTE: Provisional describes a condition, not a verification state.

**Conflict**

A situation where multiple reports provide incompatible information.

**Confidence Level**

An assessment of conflicts.

#### Confidence Level Terms how dependable a specific value is in context.

Defined levels:

***High Confidence**

Official source and no conflicts detected. Confidence*

***Medium Confidence**

Multiple non-official sources agree, but no official confirmation exists. Confidence*

***Low Confidence**

Single unverified source or conflicts detected between sources. Confidence*

NOTE: Confidence depends on factors such as source, recency, and consistency.

#### Freshness and Transparency Indicator Terms

**Publish Time**

The time When a report source originally released the information.

**Ingest Time**

When the information was first made visible to users. recorded by the system.

**Last Updated Time**

The most recent time change to the system updated event record.

**Staleness**

A condition where information is no longer current enough for its context.

**Dependability Indicator**

A marker that helps explain how reliable the displayed information. information is.

**Source Type**

**Classification Attribution***

Identification of where the origin of information (e.g., Official, Reporter, Community).

**Reliability Metadata**

The combined set of fields that explain trustworthiness, including verification status, confidence level, timestamps, and source attribution. originated.

#### System State Condition Terms

**Degraded State**

A system condition where full functionality or complete data Operation continues, but information is not available due to source failures, partial outages, missing fields, missing, delayed, or inconsistencies. incomplete.

**Graceful Degradation**

The system remains usable under Providing partial failure by showing last known valid data and clearly communicating information with clear limitations instead of failing completely.

**Silent Failure**

**A failure where incorrect or missing Overwrite***

Replacing information is presented without warning, explanation, preserving previous values or status messaging. indicating change.

**Unknown**

A value is not available and should not be inferred.

### 2.1.3 Domain Terminology in Relation to Domain Rough Sketch

**Purpose**

This section connects explains how the terms defined in terminology from Section 2.1.2 with the main themes described in the Domain Rough Sketch (2.1.1). The goal is to show how each term fits into the domain and to keep used after the documentation conceptually consistent.

### Mapping Terminology to Domain Themes

#### Distributed Sources and Conflicting Information rough sketch in Section 2.1.1. The rough sketch describes a system that collects data from multiple leagues and informal sources, where disagreements between reports are expected.

Relevant terms:

- **Conflicting** — when different sources report different values for the same data point.
- **Unverified** — data coming from a single informal source with no independent confirmation.
- **Source Type** — identifies names the origin main concerns of a report (Official, Reporter, Community) the domain in broad terms. The terminology section then gives fixed terms for those concerns so the system rest of the document can distinguish authority levels.
- **Corrected** — used when previously published data is revised after a conflict is resolved.

#### Delays describe them more precisely and Missing Updates consistently.

### From Broad Concerns to Fixed Terms

The rough sketch recognizes uses plain-language descriptions such as conflicting reports, delayed updates, degraded states, and trust. Section 2.1.2 introduces fixed terms for those conditions so later sections do not have to redefine them each time.

In that updates may arrive late or sense, Section 2.1.2 does not at all, especially replace the rough sketch. It provides a controlled vocabulary for discussing the same domain concerns in local a more exact way.

### Terminology Used for Information Status

When the document discusses the status of sports contexts.

Relevant terms:

* information, it uses the verification terms from Section 2.1.2:

- **Confirmed**
- **Unverified**
- **Developing**
- **Conflicting** - **Delayed** —
- **Corrected**

These terms are used to describe how information received well after stands at a given moment, especially when reports differ, arrive late, or change over time.

### Terminology Used for Degree of Confidence

When the event occurred. * **Developing** — time-sensitive document needs to distinguish stronger from weaker support for a reported value, it uses the confidence terms from Section 2.1.2:

- **High Confidence**
- **Medium Confidence**
- **Low Confidence**

These terms do not replace verification status. They are used alongside it when the documentation needs to express how strongly available reports support a value.

**Terminology Used for Source and Freshness Context**

When the document needs to state where information that is still being verified. * came from or how recent it is, it uses:

- **Publish Time** - **Last Updated Time** — shows
- **Source Type**
- **Reliability Metadata**

These terms are used whenever the most recent system update, helping users detect stale data. * **Publish Time** — indicates when information was first shown documentation refers to users.

**High Demand Moments**

The rough sketch highlights that reliability is most critical during playoffs, tournaments, and breaking news, when traffic and user expectations are highest.

Relevant terms:

- **Confirmed** — clearly verified information, especially important during peak moments.
- **High Confidence** — official confirmation with no detected conflicts.
- **Medium Confidence** — agreement across informal sources without official confirmation.
- **Low Confidence** — data from a single unverified source attribution, freshness, or with active conflicts.

**Clear Behavior During Degraded States**

The rough sketch states that failures must be visible and understandable the contextual data attached to users.

Relevant terms:

* a report.

**Terminology Used for System Conditions**

When the document discusses conditions affecting the platform as a whole rather than a single report, it uses:

- **Degraded State** — when full functionality or complete data is unavailable. - *Graceful Degradation — the system remains usable under partial failure and clearly communicates limits. -

*Silent Failure — presenting incorrect or missing information without warning. * **Reliability Metadata** — transparency fields such as verification status, confidence level, timestamps, and source attribution.

#### Trust as

These terms are used to describe what happens when the Main Outcome platform cannot operate under normal conditions.

### Relation to the Rough Sketch

The rough sketch identifies long-term trust as remains the broad description of the domain. The terminology section remains the central objective. Trust depends on clarity, accuracy, and timeliness.

Relevant terms:

- **Confirmed** and **Unverified** — core signals that distinguish reliable data from uncertain data.
- **Source Type** — enables source attribution so users can judge reliability.
- **Publish Time** and **Last Updated Time** — provide context about data freshness.
- **Reliability Metadata** — of truth for the overall transparency layer supporting user trust. defined terms used afterward. This section only states that relationship. It does not introduce new concepts beyond those already defined in Section 2.1.2.

## 2.1.4 Narrative

### Purpose

This narrative describes how users in Puerto Rico currently obtain sports information, why that process is unreliable, and how Tu Deporte Aquí addresses uncertainty through transparency and predictable behavior.

### Context: Local Sports Information Flow

Many local sports updates are shared through informal channels (social media posts, chats, community pages, screenshots, and word-of-mouth) rather than standardized APIs. Coverage varies by league and region, and “official” updates may arrive late or contradict earlier reports.

### Actors and Typical Scenarios

#### Fans and families

Fans want quick answers (scores, standings, schedules) while commuting or during games. Most access is mobile-first and time-sensitive.

#### Players and coaches

Participants use the platform to confirm game results, future matches, and team performance, especially when announcements are scattered across multiple sources.

#### Leagues and organizers

Organizers may provide partial information (schedule updates, venue changes) that does not propagate consistently across platforms.

#### Narrative: What Happens Today (Problem Story)

A user hears that a game has started and tries to find the score. One source posts an early score update, another source posts nothing, and a third source posts a conflicting result later. The user cannot determine which information is current, which source is more trustworthy, or whether the system is missing data. In many cases, systems fail silently—showing outdated information without indicating the last update time or confidence level.

#### Narrative: Expected System Behavior (Tu Deporte Aquí)

Tu Deporte Aquí aggregates and normalizes available sports data while explicitly communicating uncertainty.

Key behaviors: - The system displays timestamps ("last updated") for relevant data. - When information is missing, delayed, or conflicting, the system communicates that state clearly rather than failing silently. - The system can present provisional results with low-confidence indicators when appropriate. - When multiple sources disagree, the system surfaces the conflict and avoids presenting a single answer as certain.

#### Why This Matters (Reliability + Transparency)

Reliability in this domain is not only about being correct—it is also about being honest about uncertainty. The platform should help users make informed interpretations by showing what is known, what is unknown, and what is still being verified.

### 2.1.5 Events, Actions, and Composite Behaviors

This section defines the key domain events, system actions, and expected behaviors for Tu Deporte Aquí. The focus is reliability describes events and transparency when actions related to how sports information is delayed, incomplete, unverified, or conflicting.

**Domain produced, compared, and updated.**

#### Domain-Relevant Events

**E1: Report Received**

**Report Arrival**:: A new update is received from a data source (league site, reporter, community post).

**E2: Live Match State Updated**

A match transitions between states such as Scheduled, Live, Halftime, Delayed, Suspended, Final, Postponed, or Canceled.

**E3: Data Missing or Partial Coverage Detected**

Expected fields (score, status, clock/segment, etc.) are missing or incomplete.

**E4: Conflict Detected**

Two or more sources disagree about a critical value (score, status, final result).

**E5: Data Becomes Stale**

The last update timestamp exceeds a defined freshness threshold.

**E6: Correction Published**

report becomes available.

**Conflict Emergence**

Multiple reports provide incompatible values.

**Correction Arrival**

A previously published value is updated report indicates that previous information should be changed.

**Freshness Loss**

Information becomes outdated due to correction or reconciliation.

**E7: Source Outage / Unavailable Source**

lack of recent updates.

**Source Interruption**

A source fails, stops updating, or becomes unreachable.

**System Actions per Event**

For each event, the system performs actions that preserve user trust and allow future validation.

*E1 providing updates.

****== Actions (Report Received)****

- Record

**Record Report**

Consumes: * source attribution and timestamps (publish time, ingest time,

- source report
- sporting event time when available).
  - Assign verification status and confidence level.
  - Store reliability metadata required for transparency.

**E2 Actions (Live Match State Updated)**

- Apply a standardized game state model and log

Produces: * stored report * associated timestamps

**Purpose**

To make the transition time.

- Support manual override and correction logging when local updates are irregular.

**E3 Actions (Missing/Partial Data)**

- Classify coverage report available for later use.

---

**Assess Dependability**

Consumes:

- recorded report
- source trust level (Full Live, Live–Score Only, Live–Status Only, Live–Delayed).
  - Preserve last known valid values when available.

**E4 Actions (Conflict Detected)**

- Set status to Conflicting and lower confidence level.
- Preserve both values and their sources to avoid silent overwrites.
- Trigger re-validation when new related
  - other reports arrive.

**E5 Actions (Stale Data)**

- Mark data as Stale/Delayed and retain last confirmed values.
- Update transparency indicators so users can see recency clearly.

**E6 Actions (Correction Published)**

- Classify

Produces: * verification state * confidence level

**Purpose**

To evaluate how reliable the change as Corrected and preserve lightweight change history for critical fields.

- Provide an explanation note (correction/override) where applicable.

**E7 Actions (Source Outage)**

- Enter Degraded State if one information is.

---

**Compare Reports**

Consumes:

- multiple reports

Produces: * consistency or more key sources fail. - Continue serving cached/last known data with clear messaging.

### Expected System Behaviors

#### Normal Operation

- When conflict

  **Purpose**

  To determine whether reports are Confirmed and non-conflicting, display agree.

---

**Update Event Record**

Consumes:

- assessed report * current values with last record

Produces: * updated time and source type. - Confidence remains High when an official source is present and no conflicts exist. record * updated timestamp

**Purpose**

To reflect the latest known information.

---

**Mark Unknown**

Consumes:

- incomplete event record

Produces: * explicit unknown value

**Purpose**

To avoid unsupported assumptions.

---

**Preserve Change Trace**

Consumes:

- old value
- new value

Produces: * record of change

**Purpose**

To maintain traceability.

#### Unverified / Developing Information

- Allow publication when time-sensitive, but label as Developing/Unverified.
- Confidence should be Low (single unverified) or Medium (multiple agreeing non-official

sources).

- Periodically re-validate and Composite Behaviors

**Information Consolidation**

Sequence of: report arrival → record report → assess dependability → update event record

---

**Conflict Handling**

Sequence of: conflict emergence → compare reports → assess dependability → update status if confirmation arrives.

**Conflicting Information**

- Display Conflicting state and communicate that information may change.
- Do not silently overwrite values; preserve traceability to explain changes later.

**Missing or Partial Live Data**

- A match can be surfaced as live only if minimum live data exists (teams, league, in-progress status, and score OR clock, plus a last updated timestamp).
- When fields are missing, degrade coverage honestly (e.g., “Clock unavailable” or “Score not available”) and keep timestamps visible.

**Delayed / Stale Information**

- If last mark unknown

---

**Correction Handling**

Sequence of: correction arrival → assess dependability → preserve change trace → update exceeds event record

---

**Freshness Management**

Sequence of: freshness threshold, loss → assess dependability → mark as Live–Delayed unknown or Stale.

- Prefer showing last known valid data retain with a warning over showing potentially incorrect new values without context.

**Source reduced confidence**

---

**Source** Failure / Degraded State - If sources fail, Handling:: Sequence of: source interruption → freshness loss → assess dependability → continue operating using cached values. - Provide system-state messaging indicating synchronization delays or degraded coverage instead of blank screens. with reduced information

---

**Corrections**

- Publish corrections explicitly (Corrected status) and preserve revision history for critical fields.
- Users should be able to understand that a change occurred and why (correction vs conflict resolution).

**Notes for Future Testing Remarks**

These behaviors are designed to be testable through simulation in later milestones: - Conflicting score scenario - Unverified developing report scenario - Delayed update / stale threshold scenario - Partial live coverage scenario - Source outage and degraded mode scenario - Correction lifecycle scenario concepts describe the domain itself. Later sections define how the system must support or present them.

## 2.1.6 #2.1.6 Function Signatures (Pseudocode at the moment, Not Event-Tied)

These Signatures#

#These function signatures describe what information is needed are tied directly to perform key actions the action groups described in **Tu Deporte Aquí** Section 2.1.5. They stay at the domain level and what state is produced use the concepts already present in the document, rather than storage-oriented or changed. The design is intentionally modular so it can be extended later without committing to a specific architecture since we don't have one decided yet.

**Core Records (state holders)**

- SourceRegistry: configured sources and basic health metadata
- RawArchive: stored raw snapshots from sources (for traceability)
- UpdateStore: stored normalized updates with provenance
- CanonicalStore: current best-known architecture-specific types.#

Notation uses the form:

functionName : InputTypes → OutputTypes

Where multiple outputs indicate updated state (games, standings, athlete updates) * ConflictQueue: unresolved conflicts that need resolution * StatusBoard: user-facing status and reliability messages produced results.

**E1 Actions (Report Received)**

buildReliabilityMetadata : # Source Configuration

registerSource(SourceRegistry, SourceConfig) → SourceRegistry

updateSourceConfig(SourceRegistry, SourceId, SourceConfig) → SourceRegistry

removeSource(SourceRegistry, SourceId) → SourceRegistry

listSources(SourceRegistry) Type >< Publish Time >< Last Updated Time [.inserted] → List<SourceConfig>

**Fetching and Snapshot Storage**

fetchSnapshot(SourceRegistry, SourceId, FetchOptions) Reliability Metadata#

Creates the reliability metadata attached to a newly received report.

classifyVerificationStatus : Source Type >< Reliability Metadata # → (SourceRegistry, Snapshot Confirmed or FetchError)

storeSnapshot(RawArchive, Snapshot) → (RawArchive, SnapshotId)

getSnapshot(RawArchive, SnapshotId) Unverified or Developing#

Determines the initial verification status of the reported information.

assignConfidenceLevel : Source Type >< Confirmed or Unverified or Developing # → Snapshot High Confidence or None [.inserted]#Medium Confidence or Low Confidence

Determines the initial confidence level of the reported information.

**Parsing and Normalization**

normalizeSnapshot(Snapshot, ParseOptions) → (List<NormalizedUpdate>, Diagnostics)

storeUpdates(UpdateStore, List<NormalizedUpdate>) E2 Actions (Live Match State Updated)

recordLiveStateUpdate : Source Type >< Last Updated Time >< Reliability Metadata # → (UpdateStore, List<UpdateId>)

getUpdate(UpdateStore, UpdateId) Reliability Metadata#

Records the reliability context of a live-match state update.

recordManualOverride : Source Type >< Publish Time >< Last Updated Time >< Reliability Metadata # → NormalizedUpdate [.inserted]#Corrected >< Reliability Metadata

Records a manual override as a traced correction when local updates are# irregular or None [.inserted]#late.

**Canonical Merge (best-known current state)**

applyUpdate(CanonicalStore, NormalizedUpdate, MergePolicy) → (CanonicalStore, Conflict #E3 Actions (Data Missing or None, Diagnostics)

applyUpdateBatch(CanonicalStore, List<NormalizedUpdate>, MergePolicy) → (CanonicalStore, List<Conflict>, Diagnostics)

getCanonical(CanonicalStore, EntityKey) Partial Coverage Detected)#

classifyPartialCoverage : Reliability Metadata >< Last Updated Time # → CanonicalEntity Developing or None

storeCanonical(CanonicalStore, CanonicalEntity) Delayed#

Determines whether incomplete coverage should be treated as still developing# [.inserted]#or already delayed.

preserveLastKnownMetadata : Reliability Metadata # → CanonicalStore [.inserted]#Reliability Metadata

Preserves the last known reliability context when current coverage is partial# [.inserted]#or incomplete.

**Conflict Handling**

enqueueConflicts(ConflictQueue, List<Conflict>) E4 Actions (Conflict Detected)

detectConflict : Reliability Metadata >< Reliability Metadata # → ConflictQueue

listConflicts(ConflictQueue) Conflicting or Confirmed#

Determines whether two reported values remain mutually consistent or enter a# [.inserted]#conflicting state.

lowerConfidenceAfterConflict : Conflicting # → List<Conflict>

resolveConflictAuto(ConflictQueue, CanonicalStore, ConflictId, MergePolicy) Low Confidence#

Lowers confidence when a conflict remains unresolved.

revalidateConflictingInformation : Conflicting >< Source Type # → (ConflictQueue, CanonicalStore, CanonicalEntity Confirmed or ResolveError, Diagnostics)

resolveConflictManual(ConflictQueue, CanonicalStore, ConflictId, ResolutionDecision) [.removed]# → (ConflictQueue, CanonicalStore, CanonicalEntity)# Conflicting or Unverified#

Re-evaluates a conflicting report when further source information arrives.

**E5 Actions (Data Becomes Stale)**

markAsDelayed : # Reliability Indicators and Degraded Messaging

computeReliability(CanonicalStore, UpdateStore, EntityKey, ReliabilityPolicy) Metadata >< Last Updated Time [.inserted] → (ReliabilityIndicator, Diagnostics)

setSystemStatus(StatusBoard, SystemStatus) Delayed >< Reliability Metadata#

Marks information as delayed when the freshness threshold has been exceeded.

preserveLastKnownReliabilityMetadata : Delayed >< Reliability Metadata # → StatusBoard

annotateEntityStatus(StatusBoard, EntityKey, ReliabilityIndicator) Reliability Metadata#

Keeps the last known reliability context visible after data becomes stale.

**E6 Actions (Correction Published)**

recordCorrection : Reliability Metadata >< Publish Time >< Last Updated Time # → StatusBoard

buildPublicView(CanonicalStore, StatusBoard, EntityKey) Corrected >< Reliability Metadata#

Records that previously published information has been corrected and updates# [.inserted]#its reliability metadata.

explainCorrection : Corrected # → PublicView or None [.inserted]#Reliability Metadata

Associates correction status with the metadata shown to users.

**User-Facing Queries**

getGame(CanonicalStore, GameId) → GameView or None

getStandings(CanonicalStore, StandingsKey) E7 Actions (Source Outage / Unavailable Source)

enterDegradedState : Reliability Metadata # → StandingsView or None

getAthleteUpdates(CanonicalStore, AthleteId) Degraded State#

Records that the system has entered a degraded state because source support# [.inserted]#is insufficient.

maintainGracefulDegradation : Degraded State >< Reliability Metadata # → List<AthleteUpdateView>

searchCanonical(CanonicalStore, SearchQuery) Graceful Degradation#

Keeps the system usable by preserving the last known reliability context# [.inserted]#under partial failure.

avoidSilentFailure : Degraded State >< Reliability Metadata # → List<PublicView> [.inserted]#Graceful Degradation

Requires failure to be communicated through visible degraded behavior rather# [.inserted]#than silent omission.

**Maintenance Note on Scope**

#These signatures stay with the actions described in Section 2.1.5 and Extensibility Hooks

recomputeAllReliability(CanonicalStore, UpdateStore, StatusBoard, ReliabilityPolicy) [.removed]# → (StatusBoard, Diagnostics)#

rebuildCanonicalFromUpdates(CanonicalStore, UpdateStore, MergePolicy) [.removed]# → (CanonicalStore, Diagnostics)# focus on the conceptual relationship each action creates or changes. Storage structures, archives, queues, and other implementation-oriented records were removed

from this section because they belong to implementation, not to the domain-level action view.#

# 2.2 Requirements

## 2.2.1 User Stories, Epics, Features

This section organizes the main requirements-driving stories for Tu Deporte Aquí. To avoid dropping unexplained persona names on the reader, each story names the role first and then identifies the corresponding persona in parentheses. The full persona descriptions appear in Section 2.2.2.

#### Epic 1: Consolidated Puerto Rico Sports Information Feed

As As a Puerto Rico sports fan, local-sports follower, I want a centralized view of games, one place to check scores, standings, schedules, and athlete updates so that I can follow local sports without chasing multiple sources.

**Key features**

- Aggregated do not have to chase scattered posts, league pages, and chats.

Features

Consolidated results, standings, schedules, and athlete updates updates The platform groups the most consulted local-sports information in one platform * Source place so that a fan or coach can check the current state of a league without jumping between unrelated pages and posts.

Source attribution for each update * Visible displayed update Each important data item should show where it came from so that readers can tell whether they are looking at an official publication, a reporter update, or a community post.

Visible freshness indicators (last indicators Each important item should show when it was last updated timestamps)

**User stories**

> so that readers can judge whether the information is still recent enough to trust during a live event or playoff run.

User stories

- As Sebastian, As a fast-checking fan (Sebastian), I want to view the latest scores and final results for selected leagues so that I can stay updated without switching between websites. websites and social media posts.
- As Sebastian, As a fast-checking fan (Sebastian), I want each score and displayed score, result, or update to show its source so that I can judge credibility quickly when multiple channels exist. are posting different things.
- As Abiel, As an organizing coach (Abiel), I want to view schedules and standings for my league so that I can plan plan, communicate, and communicate accurately with my team. verify information without having to piece it together manually.

- As Sebastian, As a fast-checking fan (Sebastian), I want to see when each item was last updated so that I can tell if information might be outdated during whether a page reflects a live events. update, a delayed post, or stale information.

**Epic Epic 2: Conflict Detection Handling, Uncertainty, and Consistency Controls**

As User Trust Signals

As a person who depends on local sports fan, information, I want the platform to detect and handle conflicting reports show when information is conflicting, incomplete, delayed, or still developing so that I do not get misled by incorrect mistake uncertain information for a final answer.

Features

Conflict flagging and conflict visibility When two or inconsistent information.

**Key features**

- Conflict detection when more sources disagree about a score, status, result, or standing, the platform should mark the disagreement clearly instead of silently choosing one value
- Validation and synchronization rules hiding the rest.

Verified, developing, and provisional indicators The platform should distinguish between information that has already been confirmed and information that is still developing, provisional, or waiting for ingestion * Clear presentation confirmation from a stronger source.

Missing-data and degraded-state messaging When a value is unavailable, delayed, or affected by a source problem, the platform should explain what is missing and what is still known instead of showing blank space or a confusing generic error.

Trusted-source prioritization When an official or clearly stronger source exists, the platform should treat that source as the primary reference while still making conflicts and resolution outcomes

**User stories**

revision history visible when needed.

User stories

- As Sebastian, As a fast-checking fan (Sebastian), I want the platform to flag when two sources disagree on a score or result so that I know the result may do not be final.
- As Andres, I want to mark updates as preliminary or confirmed so that assume the platform can represent uncertainty responsibly. first number I see is already final.
- As Abiel, As an organizing coach (Abiel), I want the platform to prioritize verified league results when available they exist so that standings and schedules do not change due to shift because of unreliable posts.
- As Chris, I want configurable validation rules for ingestion so that consistency checks can be improved as we observe real failures.

**Epic 3: Graceful Degradation and User Trust Signals**

As a user, I want clear, informative messages when data is missing or the system is degraded so that I understand what is happening and can still make decisions.

**Key features**

- Degraded mode behavior for partial outages
- Informative user-facing messaging that explains uncertainty
- Reliability and status cues (data availability, source health)

**User stories**

early posts.

- As Sebastian, As a deadline reporter (Andres), I want to see label an update as developing, provisional, or confirmed so that I can communicate uncertainty without spreading misinformation.
- As a fast-checking fan (Sebastian), I want a clear explanation when an update is missing so that I know whether it the information is unavailable, delayed, unavailable, or still being confirmed. checked.
- As Sebastian, I want the platform to show As a simple reliability cue for key pages during major events so that I can trust what I am seeing.
- As Andres, deadline reporter (Andres), I want the platform to avoid presenting uncertain info information as definitive so that users are breaking updates do not misled by breaking-news updates. mislead the public.
- As Chris, As a reliability-focused engineer (Chris), I want standardized fallback behavior when an external source fails so that failures outages do not surface as confusing errors appear to users. users as random or inconsistent errors.

**Epic 4: 3: High-Traffic Availability During Major Events**

As As a sports fan, local-sports follower, I want the platform to remain responsive usable during playoffs playoffs, tournaments, and breaking news breaking-news moments so that I can access still get the most important information when demand is highest.

**Key features**

- Load handling spikes.

Features

Peak-event responsiveness Core pages for peak events * Performance baselines scores, standings, and scalability targets * Prioritized game status should remain responsive during periods of unusually high traffic.

Prioritized delivery of critical info (scores, standings, status)

**User stories**

essential information When load is high, the platform should keep the most important information available first, even if less important details need to wait.

Recovery visibility during stressed conditions The platform should make it possible to measure whether it stays available, how fast it recovers, and what information remains accessible during stress.

User stories

- As Sebastian, As a fast-checking fan (Sebastian), I want score and standings pages to load reliably during playoffs so that I can follow games in real time.
- As Chris, I want load and stress tests that model playoff traffic so that we can identify bottlenecks before major events. while the event is still happening.
- As Sebastian, As a fast-checking fan (Sebastian), I want the platform to still show keep showing the most essential information when traffic is extreme so that I am not blocked from updates entirely. entirely.
- As Chris, As a deadline reporter (Andres), I want the platform to stay understandable during major events so that source delays, outages, or traffic spikes do not turn live coverage into confusion.
- As a reliability-focused engineer (Chris), I want to measure availability and recovery time during peak-event simulations so that reliability improvements are measurable. work can be judged with actual evidence rather than guesswork.

**Epic 5: 4: Reliability Testing Automation and Observability**

As As a platform engineer, I want reliability testing and monitoring integrated tied into the development workflows workflow so that failures conflict handling, degraded behavior, and regressions recovery logic are caught checked before users experience them.

**Key features**

- Automated tests run into them.

Features

Automated reliability test coverage The project should include automated checks for conflict, delay, conflicting data, delayed updates, missing fields, and missing-data scenarios * Fault other situations that are common in this domain.

Fault injection for external dependencies * Metrics dependency failure The team should be able to simulate source failures and dashboards for unstable conditions to verify that fallback behavior still matches the documented expectations.

Operational metrics and monitoring views The project should track reliability-related measures such as availability, accuracy, consistency, and recovery time so that improvements and regressions can be seen over time.

User stories

**User stories**

- As Chris, As a reliability-focused engineer (Chris), I want automated tests that simulate conflicting data sources so that system behavior stays predictable when sources disagree.

disagree.

- As Chris, As a reliability-focused engineer (Chris), I want automated tests that simulate delayed updates and missing data so that user-facing behavior remains stays clear and informative. when the incoming information is incomplete.
- As Chris, As a reliability-focused engineer (Chris), I want fault injection tests against external dependencies so that we can verify fallback strategies and error-handling logic. logic before real outages happen.
- As Chris, As a reliability-focused engineer (Chris), I want CI workflows to run reliability tests regularly so that testing covers coverage includes stress and failure conditions, not only normal operation. operation.
- As Chris, As a reliability-focused engineer (Chris), I want dashboards for accuracy, consistency, availability, and recovery time so that we can track reliability improvements over time. see whether the system is actually improving.

### 2.2.2 Personas

**Persona A: Sebastian Rivera, #The personas below ground the Superfan user stories in Section 2.2.1. They are meant**

to capture habits, pressure, context, and acceptable tradeoffs in this domain, not just a short list of goals and pain points.#

=== Sebastian Rivera "The Superfan"

#Background: Sebastian Rivera is a 22-year-old Puerto Rico sports fan from Bayamón who follows local leagues weekly and the way some people follow national teams. He checks scores during games and playoffs. He values speed games, refreshes standings during playoffs, and clarity, keeps group chats alive with "cuanto va el juego?" messages. Most of his sports information comes from a messy mix of league posts, Instagram stories, and whatever link someone drops in a chat. When everything lines up, he gets frustrated when different what he needs fast. When sources disagree conflict or updates arrive late.

**Goals**

- Quickly see late, confusion starts immediately.#

Sebastian does not want deep analysis. He wants to know what happened, what# is happening now, and whether the information in front of him deserves trust, especially when a big game [.inserted]#is on and everyone around him is checking the same thing.

Age: 22 years old. Role: College student and weekly local-sports follower. Interests: Sebastian likes staying connected to local sports culture. He# watches games with friends, follows team accounts, and cares about standings changes because they become instant conversation in chats and on campus. He likes quick, clear information and gets impatient when he has to hunt for what [.inserted]#should have been a simple score update.

#Goals: Sebastian wants one fast place to check scores, final results, standings, and key updates in one place * Know updates. He wants to know whether information is current and trustworthy enough to trust during big events * Avoid confusion when reports conflict

**Pain points**

- Different pages and posts report different scores or update at different times
- During playoffs, sites lag or go down exactly when he needs them most
- When something is missing, he does not know if it is an error or just not posted yet

**Persona B: Coach Abiel Rullan, the Team Organizer**

Abiel manages a local team and cares about schedules, standings, live games and accurate public reporting of results. playoffs. He also wants the platform to reflect reality say clearly when something is uncertain, instead of leaving him to guess whether the problem is the source, the system, or the event itself.

#Problems: Puerto Rico sports updates are scattered across platforms and avoid spreading incorrect information about teams often inconsistent. Sebastian runs into different sources reporting different scores, missing context, or players.

**Goals**

- Confirm schedules posting results late. During playoffs and standings are accurate for his league
- Catch major events, the places he checks most often become slow or unreliable, and correct errors quickly when something looks wrong
- Share trustworthy is missing he cannot tell if it has not been posted yet or if something broke.

Quotes: "I do not want to search five places just to confirm the score." "If the info is not reliable during playoffs, what is the point?"

Solutions: Feature - Consolidated Results and Standings: A centralized place to view# scores, standings, and key updates without hopping between sources. Feature - Freshness and Source Labels: Show where the info came from and when it was last updated so Sebastian can judge reliability quickly. Feature - Degraded Messaging: Clear explanations when updates are delayed or [.inserted]#missing so uncertainty does not look like an error.

# Coach Abiel Rullan "The Team Organizer"

Background: Abiel Rullan is a local coach and team organizer who manages schedules, communicates with players and parents

**Pain points**

- Updates are often delayed parents, and he has cares deeply about results being reported correctly. For Abiel, sports information is not just something to double-check multiple sources
- Conflicting reports create arguments check for fun. It affects planning, transportation, attendance, and confusion among teams team morale. When standings or schedules are wrong online, he ends up answering questions, calming people down, and fans
- There correcting the situation by hand.#

Abiel can tolerate honest delay. What frustrates him# is having no clear way to tell if [.inserted]#whether something is verified, provisional, or simply outdated.

Age: 34 years old. Role: Coach and league participant (organizer/communicator). Interests: Abiel values structure and clarity. He wants schedules and# standings that match reality and a way to communicate accurate information with confidence. He frequently double-checks postings because he has learned [.inserted]#that one random post can spread faster than an official correction.

Goals: Abiel wants standings and schedules that reflect what actually# happened, not what one account posted first. He wants a clean way to separate confirmed information from early or doubtful reports. He also wants to spot conflicts quickly so he can avoid repeating bad information to players, [.inserted]#families, and other teams.

#Problems: Updates can be delayed, incomplete, or inconsistent across sources. Conflicting reports create disputes among teams and fans. Abiel often has to verify information manually because he cannot tell which source is authoritative or whether a result is verified or provisional

**Persona C: Andres Perez, final.**

Quotes: "If# the [.removed]#Sports Reporter standings are wrong, I am the one who has to explain it to everyone." "I just need a clear way to know what is confirmed and what is still pending."

Solutions: Feature - Verified vs Provisional Indicators: A domain-level signal that# distinguishes confirmed results from developing reports. Feature - Conflict Flagging: Highlight disagreements between sources so Abiel knows not to treat the information as final. Feature - Prioritize Trusted Sources: When official league results exist, [.inserted]#they should be treated as the strongest reference.

=== Andres covers Puerto Rico Perez "The Deadline Reporter"

Background: Andres Perez is a 26-year-old# sports reporter and community updater who posts rapid updates, especially updates during tournaments and breaking news. moments. He needs is used to working with incomplete information at first. Someone sends a platform that can partial score, a post goes up without details, and corrections arrive later. Andres cares about speed, but he cares even more about not spreading misinformation. He wants a clean way to represent uncertainty cleanly #while the story is still moving.

#Andres also knows that the platform will get blamed for confusion even when details the underlying sources are the real problem. If the platform becomes slow during major events or fails when a source goes down, the audience will blame the platform first.

Age: 26 years old. Role: Sports reporter / community updater. Interests: Andres likes live coverage, community engagement, and being first# without being wrong. He pays attention to the difference between what is known now and what is still developing. He also cares about whether the platform stays readable under pressure, because high-traffic moments are when his work [.inserted]#matters most.

#Goals: Andres wants to publish or

**Goals**

- Publish and reference updates without causing misinformation
- Make it clear when an update overstating what is preliminary vs confirmed
- Keep coverage known. He wants the platform to mark developing information clearly, keep conflicts visible, and stay stable enough that live coverage does not collapse during high-traffic moments

**Pain points**

* the most important events.

Problems: Early reports may be are often incomplete, but the public wants immediate updates * A single wrong post updates. Conflicting sources can spread fast if the platform does not handle conflicts well * misinformation quickly. When systems degrade, users often receive get confusing or unhelpful error messages

**Persona D: Chris Ramos, errors instead of clear explanations, and trust**

drops fast. During playoffs and high-traffic events, performance issues become visible immediately.

Quotes: "I can post fast, but I need a way to show what is confirmed and what is still developing." "If the site goes down during the biggest game, people will not trust it again."

Solutions: Feature - Preliminary vs Confirmed Updates: A way to represent developing# information responsibly. Feature - Standardized Fallback Behavior: Clear behavior when sources fail so the platform stays understandable during outages. Feature - Reliability Testing and Monitoring Foundation: Stress and failure [.inserted]#scenarios modeled early so peak-event reliability is not a surprise.

# Chris Ramos "The Platform Engineer Engineer"

Background:# Chris Ramos is responsible for reliability the team member who gets pulled in when the platform behaves differently under pressure than it did in a calm demo. He watches what happens when sources conflict, when one feed stops updating, when traffic jumps, and quality of when recovery does not happen the sports information platform. same way twice. Chris is less interested in whether one screen looks polished and more interested in whether the same domain situation leads to the same system behavior every [.inserted]#time.

Chris expects this project to operate in a messy environment.# He focuses assumes that sources will disagree, some updates will arrive late, and outages will happen. What matters to him is whether the platform reacts in a way that can be [.inserted]#understood, tested, and improved.

Age: 28 years old. Role: Platform engineer focused# on reliability and quality. [.inserted]#Interests: Chris cares about# predictable behavior under failures, automation in behavior, repeatable testing, observability, and clear user-facing status.

**Goals**

- Prevent incorrect fallback logic. He wants the project to expose where things fail, not hide those failures behind vague messages or outdated information from being shown as “final”

- Keep silent assumptions. He also cares about having evidence when someone says the system available during peak events
- Standardize is more reliable than it was before.#

#Goals: Chris wants reliability tests rules that can be tested, fallback behavior that stays consistent under failure, and integrate metrics that show whether conflict handling, freshness handling, and recovery behavior are getting better or worse over time. He also wants stress conditions to be modeled before users find them into CI

**Pain points**

- Most tests pass first.#

Problems: If source failures are handled differently across pages, users lose# trust and developers lose time. If reliability behavior is not covered by tests, regressions slip in normal conditions but quietly. If the team cannot see accuracy, availability, consistency, and recovery trends, then reliability arguments [.inserted]#turn into opinion instead of evidence.

Quotes: "If we cannot test it under failure, we# do not cover really know how it behaves." "I do not want the first real outage to be the first time we learn the [.inserted]#fallback logic is inconsistent."

Solutions: Feature - Automated Reliability Tests: Simulate conflict, delay, missing-data,# and outage scenarios before users encounter them. Feature - Fault Injection and Recovery Checks: Verify that external dependency failures trigger the documented fallback behavior. [.inserted]#Feature - Monitoring# and stress * External sources fail or behave unpredictably * It is hard to measure trust-impacting issues without good observability Reliability Metrics: Track availability, accuracy, consistency, and recovery time so changes can be judged with evidence.

### 2.2.3 Domain Requirements

**Purpose**

This section defines the domain requirements for **Tu Deporte Aquí**. These requirements are derived from observations of the sports information domain in Puerto Rico and capture the essential phenomena and concepts that the system must faithfully simulate. The domain properties listed below form the basis for these requirements.

**Observed Domain Properties**

The following properties have been observed simulate, grounded in observations from the domain of local Puerto Rico sports information:

- DP-1: Official league sources (e.g., league websites, official announcements) are considered more authoritative than informal sources such as social media posts or news outlets.
- DP-2: When multiple independent unofficial sources report the same value (e.g., a game score), confidence in that value increases, but it remains unverified until confirmed by an official source.
- DP-3: Corrections to game results, standings, or athlete information occur description (section 2.1) and must be reflected in historical records.

- DP-4: Information becomes less reliable over time; after a certain period without updates, it is considered stale.
- DP-5: Conflicts arise when different sources report different values for the same game attribute (score, status, etc.).
- DP-6: Sources may become temporarily or permanently unavailable, affecting the reliability of data that depended on them.
- DP-7: Games progress through a set of states (scheduled, live, halftime, final, postponed, cancelled) that must be tracked. epics and user stories (section 1.2).

**Domain Requirements**

The following requirements specify what the system must do to faithfully simulate the domain concepts and phenomena listed above. Each requirement is traceable to one or more domain properties.

| | |
|---|---|
| **R-DOM-1 (Source [.removed]#Authority)** | The system# Authority - Automated Sources):: External data sources shall assign a be assigned trust level to each configured source levels based on its type (official, news, social). type: Official sources (league websites, official announcements) shall have high trust; News outlets and aggregators shall have medium trust; Social media and user-generated sources shall have the highest low trust. |
| **[.inserted]#R-DOM-1a (Source Authority - Manual Input)** | Data manually entered by authorized users shall be assigned# trust level. (DP-1) levels based on role: League representatives and system administrators shall have high trust; Reporters and coaches/organizers shall have medium trust; Authorized community members shall have low trust. Manual entries shall be properly attributed and timestamped by role. |
| **R-DOM-2 (Conflict Detection)** | The system shall detect when two or more sources report different values for the same game attribute (e.g., score, status) and mark that attribute as conflicting. (DP-5) Multiple sources reporting the same value (even with different timestamps) do not constitute a conflict. |
| **R-DOM-3 (Conflict Preservation)** | Preservation and Resolution):: When a conflict is detected, the system shall preserve all reported values and their source attributions. The system Three timestamp types shall not automatically overwrite be tracked: Event Time (when the canonical value; resolution event occurred in the domain), Ingestion Time (when the data entered |

| | |
|---|---|
| | the system), and Reported Time (when the source reported it). All reported values shall follow defined policies (e.g., official be visible together, with the highest-trust source preference). (DP-1, DP-5) and most recent reported time emphasized. The canonical value shall not be automatically overwritten; all conflicting values are retained for manual or rule-based resolution. |
| **R-DOM-4 [.removed]#(Provisional Data)** | If only unofficial# (Unverified Data State):: Data from non-high-authority sources report a value, the system shall mark that data as provisional. Provisional data may be displayed but must placed in the unverified verification state, indicating it has not been confirmed by an official source. Unverified data shall be clearly distinguished from confirmed data. (DP-2) |
| **R-DOM-5 (Confidence Building)** | data (See R-INT-9). When multiple independent unofficial sources report the same value for a game attribute, the system may increase the confidence level associated with that data, but it shall remain marked as unverified until an official source confirms it. (DP-2) such data, the verification state shall transition from unverified to confirmed. |
| **[.inserted]#R-DOM-5 (Confidence Calculation)** | Confidence levels for data shall be calculated from verification state, source type authority, and staleness according to the following rules: If an official source exists and there are no conflicts and data is not stale, confidence shall be High. If multiple unofficial sources agree and there are no conflicts and data is not stale, confidence shall be Medium. If data comes from a single unverified source, or if there are unresolved conflicts, or if all sources are inactive, or if data is stale, confidence shall be Low. Otherwise, confidence defaults to Low.# |
| **R-DOM-6 (Corrections)** | (Corrections and Audit Trail):: When an official source issues a correction to a previously published value, the system shall update the canonical value and retain the previous value in create an audit trail, along trail entry with the previous value, reason (e.g., “official correction”) "official correction"), source, and timestamp. (DP-3) |

| | |
|---|---|
| **R-DOM-7 (Staleness)** | The system shall track the last update time for each game attribute. If the time since attribute and mark data as stale according to the last update exceeds a configurable freshness threshold (e.g., 30 minutes for following thresholds: For live game scores, 24 hours for standings), if no update is received within 15 minutes, the data shall be marked as stale. (DP-4) For game state transitions (R-DOM-9), if no state change is received within 30 minutes past the scheduled game start time, or within 3 hours after the game is marked Live, the state shall be marked as stale. For standings, if no update is received within 24 hours, the data shall be marked as stale. For schedules, if no update is received within 48 hours, the data shall be marked as stale. When data transitions to stale state, it shall trigger confidence recalculation (R-DOM-5) and the stale status shall be communicated to dependent components via staleness indicators (see R-INT-9). |
| **R-DOM-8 (Source Unavailability)** | If a source ceases to provide updates (e.g., repeated fetch failures), the system shall consider mark that source inactive and adjust the reliability as inactive. The confidence level of data that depended solely on it. Data from inactive sources shall be treated with lower confidence. (DP-6) reflect source availability via the staleness and confidence mechanisms defined in R-DOM-5 and R-DOM-7. |
| **R-DOM-9 (Game State Transitions)** | The system shall model game states (scheduled, live, halftime, overtime, delayed, suspended, final, postponed, cancelled) and record each enforce valid state transitions. Valid transitions are: Scheduled → Live (at start time); Live → Halftime (at regulation end); Halftime → Live (when break ends); Live → Overtime (if tied at regulation end); Live/Overtime → Final (at game completion); Live → Delayed (for technical interruptions); Live → Suspended (for indefinite pause); Suspended → Live (when resumed); Scheduled → Postponed (before starting); Scheduled → Cancelled (before starting); any state → Final (for corrections). Each state transition shall be recorded with a timestamp timestamp, source, and source. (DP-7) reason. |

| | |
|---|---|
| **R-DOM-10 (Audit [.removed]#Trail)** | The system shall maintain an audit trail# Trail for all Critical Data):: All changes to critical game data (score, status, final result) including the must be recorded in an audit trail. Each entry shall include: previous value, new value, source, source of the change (automated source or user), timestamp, and reason (e.g., "official correction", "user update"). |
| **[.inserted]#R-DOM-11 (Conflict Logging)** | When the system detects conflicting reports from multiple sources, a conflict log entry shall be created with: both conflicting values, the sources reporting each, the timestamps of each report (event time), and whether the conflict has been manually resolved. Conflict logs shall be retained# for audit and analysis purposes. |
| **[.inserted]#R-DOM-12 (User Attribution)** | All manual changes to domain data shall be attributed to the authenticated user who performed the change. The audit trail must record the user ID (or role if applicable) along with each manual# change. (DP-3) |

### 2.2.4 Interface Requirements

#### Purpose

This section defines the interface requirements for **Tu Deporte Aquí**. These requirements specify how the system interacts with the external world to acquire, initialize, and update the representation of domain phenomena. They cover both automated data ingestion from external sources and manual input by authorized users.

#### Shared Phenomena

The following domain phenomena must be represented inside the system and kept synchronized with the outside world:

- Game scores, statuses, and schedules reported by various sources (league websites, social media, news outlets, direct reports)
- Corrections issued by official leagues
- Source availability and health
- User-initiated updates (e.g., by reporters or league officials)

#### Interface Requirements

| | |
|---|---|
| **R-INT-1 (Automated Ingestion)** | The system shall provide means to periodically fetch data from configured external sources (e.g., league websites, RSS feeds, APIs, social media |

| | |
|---|---|
| | streams). The fetching mechanism shall respect source-specific constraints such as rate limits and update frequencies. |
| **R-INT-2 (Manual Data Entry)** | The system shall provide an interface for authorized users (e.g., reporters, league representatives) to manually enter or update game information, including scores, statuses, and corrections. Manual entries shall be attributed to the user and timestamped. Trust levels for manual entries shall follow R-DOM-1a based on user role (league representative, reporter, etc.). |
| **R-INT-3 (Source Configuration)** | The system shall provide means a user interface for administrators to configure data sources, including their sources. Configuration data (source type, trust level, fetch schedule, and access schedule) shall be persistently stored in the database. Access credentials (if any). Changes to source shall be securely encrypted. All configuration changes shall be logged. logged with timestamp and user attribution. |
| **R-INT-4 (Data Initialization)** | When a new league, team, or game is added to the system, the The system shall provide means an interface to initialize its representation data for new leagues, teams, or games. Initial data loaded from existing sources or manual entry. Initial data entry shall include at least be persistently stored in the database. Required initial data includes teams, scheduled dates, and any available historical results. |
| **R-INT-5 (Update on Domain Event)** | When a domain event occurs (e.g., a game ends, a score changes), the system must be updated accordingly. This can happen through either automated detection via source polling (polling or webhooks, webhooks) or manual input by authorized users. The system Updates shall ensure that updates are reflected be persistently recorded in the canonical data store with appropriate provenance metadata (source, timestamp). |
| **[.removed]#R-INT-6 (Correction Handling)** | When a correction is identified (either through automated detection of a discrepancy or manual notification), the system shall provide means to apply the correction, preserving the previous value in the audit trail. The correction shall be |

| | |
|---|---|
| | attributed to the source (e.g., official league, user) and timestamped.# timestamp, event time per R-DOM-3). |
| **R-INT-7 (Conflict Notification)** | When the system detects conflicting reports, it shall provide means to notify appropriate personnel (e.g., via dashboard, browser alerts, and mobile notifications (if installed as a dashboard or alerts) PWA, see R-MAC-14) so that manual resolution can occur if automated resolution policies are insufficient. |
| **R-INT-8 (Source Health Monitoring)** | (Status Dashboard):: The system shall monitor the availability and responsiveness of configured sources. It shall log failures and provide visibility into an administrative dashboard displaying real-time status information: source health (e.g., via a status dashboard) for administrators. |
| **[.removed]#R-INT-9 (User Attribution)** | All manual changes to domain data# (success/failure counts, response times, last success timestamp), request rates and response times per endpoint, conflict counts and resolution status, and system resource usage (CPU, memory, disk). The dashboard shall be attributed to the authenticated user who performed the change. updated at least every 30 seconds. |
| **R-INT-9 (Unverified Data Display)** | The system shall record the user ID along display unverified data (from non-high-authority sources) with the a clear visual indicator distinguishing it from confirmed data. Unverified data shall include visible source attribution and indicate that it may change in the audit trail. when official confirmation arrives. |

### 2.2.5 Machine Requirements

#### Purpose

This section defines the machine (non-functional) requirements for **Tu Deporte Aquí**. These requirements address performance, scalability, reliability, security, and operational aspects of the system. They are stated in measurable terms where possible; some thresholds are initial estimates and will be refined as the project progresses and more data becomes available.

#### Notes on Current Status

The requirements below represent the current understanding of system constraints and quality goals. As the project moves into implementation, some values (e.g., exact concurrency numbers,

response time targets) may be adjusted based on load testing, user research, and infrastructure decisions. Items marked with “TBD” indicate areas requiring further investigation.

**Machine Requirements**

| | |
|---|---|
| **R-MAC-1 (Availability)** | The system shall be available to users 24/7 with a target uptime of 99.5% (excluding planned maintenance). Planned maintenance windows shall be announced at least 24 hours in advance. |
| **R-MAC-2 (Response Time - Normal Load)** | Under normal load (concurrent users ≤ 100), the system shall respond to user-facing requests (e.g., loading game details, standings) within 200 milliseconds for 95% of requests. |
| **R-MAC-3 (Response Time - Peak Load)** | During peak events (e.g., playoffs, with concurrent users up to 1000), the system shall respond to critical requests (scores, game status) within 500 milliseconds for 90% of requests. Non-critical features may experience higher latency. |
| **R-MAC-4 (Scalability)** | The system shall be designed to handle an increase in concurrent users up to 2000 without significant degradation (response times may increase but remain below 2 seconds for critical requests). The architecture shall support horizontal scaling of stateless components. |
| **R-MAC-5 (Data Freshness)** | The system shall update live game scores from sources at least every 5 to 10 minutes during games. games, providing a safety margin before the 15-minute staleness threshold (R-DOM-7). For non-live data (standings, schedules), updates shall occur at least once per day. (Freshness thresholds for staleness marking are defined in R-DOM-7.) |
| **[.removed]#R-MAC-6 (Conflict Logging)** | The system shall log all detected data conflicts, including the entities involved, the conflicting values, sources, and timestamps. Logs shall be retained for at least 30 days for analysis and auditing.# |
| **R-MAC-7 (Error Logging)** | Logging and Retention):: The system shall log all errors occurring in the machine (e.g., source fetch failures, parsing errors, database connectivity issues) with sufficient detail (timestamp, source, machine-level detail: timestamp, source identifier, error type) to |

| | |
|---|---|
| | facilitate debugging. type/code, and stack trace or diagnostic information. Error logs shall be persistently stored and retained for at least 90 days. |
| **[.removed]#R-MAC-8 (Monitoring and Metrics)** | The system shall expose the following metrics, which shall be available via a monitoring dashboard and/or a metrics endpoint:#<br>• Source health (success/failure counts, response times, last success timestamp)<br>• Request rates and response times per endpoint<br>• Conflict counts and resolution status<br>• System resource usage (CPU, memory, disk) |
| **R-MAC-9 (Graceful Degradation)** | If one or more data sources become unavailable, the system shall continue to serve cached data for affected entities. data. The system shall clearly indicate (via user-facing messages) that flag cached data may be stale or partial. with staleness indicators (according to R-DOM-7) to communicate data age to dependent components. |
| **R-MAC-10 (Data Retention)** | The system shall retain canonical game data indefinitely for historical reference. Audit logs and conflict logs shall be retained according to policy (at least 30 days). Raw source snapshots may be retained for a shorter period (e.g., 7 days) for debugging. |
| **R-MAC-11 (Security)** | (Security and Authentication):: All communication between the system and channels with external sources shall be over HTTPS. use HTTPS with certificate verification. Access to manual data entry and administrative functions shall require bearer tokens, API keys, or session tokens issued by an authentication and authorization. User passwords, service. Sensitive data (credentials, authentication tokens) shall be encrypted at rest using AES-256. Local authentication credentials, if any, stored, shall be hashed and salted. using bcrypt or equivalent with salt. |
| **R-MAC-12 (Backup and Recovery)** | The system's database shall be backed up daily, with backups retained for at least 30 days. Recovery procedures shall be documented and |

| | |
|---|---|
| | tested at least once per quarter. |
| **R-MAC-13 (Testability)** | The system shall be designed to support automated testing of reliability scenarios, including source failures, conflicting data, and stale data. Test harnesses shall allow simulation of these conditions. |
| **R-MAC-14 [.removed]#(Performance Testing)** | Performance benchmarks (response times, throughput) shall be validated through load testing before each major release.# (Progressive Web Application and Service Worker Support):: The system shall meet support installation as a progressive web application (PWA) and registration of service workers. Service workers shall receive and handle push notifications using the targets defined in R-MAC-2 Push API with VAPID key authentication. On mobile platforms, PWA installation shall be required to enable push notification receipt; on desktop platforms, service workers may operate without PWA installation. |
| **[.inserted]#R-MAC-15 (Responsive Layout)** | The application markup# and R-MAC-3 under simulated peak load. stylesheets shall implement responsive design techniques to adapt the layout and display across different viewport widths. The application shall prioritize mobile viewport dimensions and maintain usability on screens with varying sizes from small mobile devices to desktop displays. |

**Open Items**

The following aspects require further investigation and will be refined in later milestones: investigation:

- **Concurrency validation**: Precise concurrency numbers (R-MAC-2, R-MAC-3, R-MAC-4) should be validated through load testing against expected user base and traffic patterns. Current targets (100/1000/2000 concurrent users) are estimates pending infrastructure measurement.
- **Response time calibration**: Response time targets (R-MAC-2, R-MAC-3) may be adjusted based on actual network conditions conditions, infrastructure capacity, and infrastructure. end-to-end latency measurements. No baseline metrics currently collected.
- **Log retention policy**: Log retention periods may be extended or shortened based on (R-MAC-7, R-MAC-10) require formalization of compliance requirements, regulatory obligations (if any), and debugging needs. audit trail lifecycle management.
- Security requirements will be expanded as authentication **Security** and authorization expansion: R-MAC-11 specifies authentication mechanisms (bearer tokens, HTTPS, encryption, hashing) and CSP headers are designed. configured. However, authorization model (role-based

access control, API key rotation, secrets management, cache poisoning hardening) requires detailed design and implementation. Ye

# 2.3 Implementation

## 2.3.1 Selected Implementation Fragments

**Purpose**

This section presents representative implementation concepts that illustrate how Tu Deporte Aquí could satisfy key reliability and transparency requirements identified during Milestone 1. These fragments are conceptual descriptions rather than complete implementations and are intended to clarify expected system behavior.

**Scope and Assumptions**

The fragments described in this section are illustrative only and may be implemented differently depending on the final technology stack and architectural decisions. No full system implementation is developed during Milestone 1. The focus is on demonstrating how the system could manage uncertain, delayed, missing, or conflicting sports data while maintaining transparency for users.

**Fragment 1: Representation of Data with Reliability Metadata**

The system should represent each piece of sports information together with contextual metadata that describes its reliability and origin. Instead of storing only the value itself (such as a score or schedule), the system should also maintain information about the source that provided the data, when it was observed, when it was ingested into the system, and the level of confidence associated with it.

Each data item should include a status indicator that communicates whether the information is confirmed, provisional, conflicting, or missing. This allows users to understand not only what the data is, but also how trustworthy it is. Including timestamps for both observation and ingestion supports transparency by revealing possible delays between real-world events and system updates.

**Fragment 2: Conflict Detection and Resolution Strategy**

Because Tu Deporte Aquí relies on multiple informal or partially reliable sources, conflicting information is expected. The system should include mechanisms to detect when multiple sources report different values for the same event. When such conflicts occur, the system should apply a predefined and documented resolution policy to select a displayed value.

The resolution policy may consider factors such as source reliability, recency of information, or agreement among sources. Importantly, even when a value is selected, the system should explicitly communicate that a conflict exists rather than silently choosing one value. This maintains user trust and supports transparency.

**Fragment 3: Data Freshness and Staleness Indicators**

Users must be able to evaluate how current the displayed information is. The system should

therefore calculate and expose freshness indicators based on timestamps associated with the data. If information has not been updated within an expected time window, it should be marked as stale.

Displaying “last updated” timestamps allows users to interpret whether the information reflects real-time conditions or delayed reporting. Different types of data may have different acceptable freshness windows depending on whether they represent live scores, schedules, or standings.

**Fragment 4: User Interface Communication of Uncertainty**

The user interface should consistently communicate uncertain system states using clear and predictable visual indicators. Missing data should never appear as empty or broken content. Instead, the interface should display explanatory placeholders such as “Not reported yet.” Provisional information should include visible indicators that it may change, and conflicting data should be labeled accordingly with the option to view source details.

All displayed information should include attribution and update timing whenever possible. This approach ensures that users remain informed about system reliability and reduces confusion during periods of incomplete or inconsistent data reporting.

**Traceability to Requirements**

These fragments directly support the reliability and transparency needs identified earlier in the documentation. By making uncertainty visible and predictable, the system improves user trust and allows validation activities to verify whether behaviors align with documented expectations.

**Notes**

The selected fragments are intentionally small conceptual anchors rather than detailed technical specifications. They provide guidance for future implementation phases while remaining aligned with the documentation-focused objectives of Milestone 1.

# 3 Analytic part

## 3.1 Concept Analysis

**Purpose**

This section analyzes the key conceptual elements that define the behavior, structure, and reliability characteristics of Tu Deporte Aquí.

The analysis focuses on how uncertainty, conflicting information, and informal data sources influence system design decisions.

**Core Concepts**

**Sports Information Entity**

Sports information represents data related to games, teams, leagues, standings, scores, schedules, and events.

This information may originate from multiple sources with varying reliability levels.

**Data Sources**

Data sources include:

- League websites
- Social media updates
- News outlets
- Community reports

Sources may differ in accuracy, timeliness, and structure.

**Reliability State**

The reliability state represents the confidence level of a piece of data.

Possible states include:

- Confirmed
- Provisional
- Conflicting
- Missing
- Stale

**Timestamp and Freshness**

Each data element includes temporal metadata indicating when the information was last updated.

Freshness indicators help users interpret reliability.

### Concept Relationships

#### Source to Data Relationship

Multiple sources may provide information about the same event.

The system must support:

- Source attribution
- Conflict detection
- Priority rules

### Reliability Interpretation

Reliability states influence:

- User interface indicators
- System decision logic
- Validation behavior

### User Trust Impact

Transparent communication of uncertainty improves user trust and reduces misinterpretation of data.

### Conceptual Constraints

#### Informal Data Environment

Tu Deporte Aquí operates in an environment where:

- Official APIs may not exist
- Data may be delayed
- Conflicts are common

System concepts must accommodate these constraints.

### Mobile-First Context

Concept design must prioritize:

- Compact information display
- Clear status indicators
- Fast interpretation

**Summary**

Concept analysis provides a structured understanding of the domain elements that influence system reliability, transparency, and behavior.

These concepts guide later modeling and validation activities.

# 3.2 Validation and Verification

**Purpose**

This section defines how the concepts, assumptions, and reliability strategies proposed for Tu Deporte Aquí will be evaluated during Milestone 1. Since this milestone focuses on documentation rather than implementation, validation emphasizes conceptual correctness, clarity, and alignment with project goals rather than executable testing.

Validation ensures that the proposed system behavior addresses real-world conditions of local Puerto Rico sports data, including delayed updates, conflicting sources, and limited official APIs. Verification confirms that documentation artifacts are internally consistent and correctly structured.

**Validation Approach**

Validation during Milestone 1 focuses on confirming that the proposed concepts meet stakeholder expectations and realistic operating conditions.

**Concept Validation**

The reliability and transparency strategies are evaluated by analyzing whether they:

- Address delayed, missing, and conflicting information scenarios
- Reflect informal data environments common in Puerto Rico sports
- Provide predictable and understandable system behavior
- Align with user expectations for trust and clarity

**Scenario-Based Evaluation**

Concepts are examined using representative scenarios such as:

- Conflicting scores reported by multiple sources
- Missing updates during live events
- Delayed standings publication
- Partial data availability from unreliable sources

These scenarios help determine whether the proposed behaviors remain consistent and reasonable.

#### Verification Approach

Verification ensures the documentation itself is correct, complete, and internally consistent.

##### Documentation Consistency

Verification activities include:

- Cross-checking terminology across sections
- Ensuring alignment between needs, ideas, and concepts
- Confirming traceability between sections of the A3 expansion
- Reviewing structural correctness of AsciiDoc hierarchy

##### Structural Verification

The documentation structure is verified by:

- Confirming correct section and subsection levels
- Ensuring files are placed in the correct repository folders
- Validating successful compilation through the AsciiDoc workflow

##### Expected Outcomes

By completing validation and verification activities during Milestone 1, the project establishes:

- Confidence in conceptual correctness
- Clear alignment with stakeholder needs
- Reliable documentation foundation for implementation phases
- Reduced risk of misunderstanding before development begins

This approach supports a documentation-first engineering methodology and prepares the project for future technical validation stages.

### 3.2.1 Validation and Verification

#### Purpose

This section defines how the concepts, assumptions, and reliability strategies proposed for Tu Deporte Aquí will be evaluated during Milestone 1. Since this milestone focuses on documentation rather than implementation, validation emphasizes conceptual correctness, clarity, and alignment with project goals rather than executable testing.

Validation ensures that the proposed system behavior addresses real-world conditions of local Puerto Rico sports data, including delayed updates, conflicting sources, and limited official APIs. Verification confirms that documentation artifacts are internally consistent and correctly structured.

## Validation Approach

Validation during Milestone 1 focuses on confirming that the proposed concepts meet stakeholder expectations and realistic operating conditions.

### Concept Validation

The reliability and transparency strategies are evaluated by analyzing whether they:

- Address delayed, missing, and conflicting information scenarios
- Reflect informal data environments common in Puerto Rico sports
- Provide predictable and understandable system behavior
- Align with user expectations for trust and clarity

### Scenario-Based Evaluation

Concepts are examined using representative scenarios such as:

- Conflicting scores reported by multiple sources
- Missing updates during live events
- Delayed standings publication
- Partial data availability from unreliable sources

These scenarios help determine whether the proposed behaviors remain consistent and reasonable.

## Verification Approach

Verification ensures the documentation itself is correct, complete, and internally consistent.

### Documentation Consistency

Verification activities include:

- Cross-checking terminology across sections
- Ensuring alignment between needs, ideas, and concepts
- Confirming traceability between sections of the A3 expansion
- Reviewing structural correctness of AsciiDoc hierarchy

### Structural Verification

The documentation structure is verified by:

- Confirming correct section and subsection levels
- Ensuring files are placed in the correct repository folders
- Validating successful compilation through the AsciiDoc workflow

#### Expected Outcomes

By completing validation and verification activities during Milestone 1, the project establishes:

- Confidence in conceptual correctness
- Clear alignment with stakeholder needs
- Reliable documentation foundation for implementation phases
- Reduced risk of misunderstanding before development begins

This approach supports a documentation-first engineering methodology and prepares the project for future technical validation stages.

### 3.2.2 System Testing Perspectives

#### Purpose

This section describes testing strategies that will be applied during future development stages of the Tu Deporte Aquí platform. This section outlines testing approaches that will evaluate the behavior of the # [.inserted]#system once implementation begins.

These testing strategies focus on ensuring that the platform remains reliable, responsive, and easy to use for sports fans accessing information such as teams, athletes, standings, game schedules, and sports news.

#### Testing Approach

Testing for Tu Deporte Aquí focuses on evaluating both the technical performance of the system and the usability of the user interface. These testing perspectives help ensure that users can access sports information quickly and interact with the platform without difficulty.

#### Performance Testing

Performance testing evaluates how the system behaves when multiple users access the platform simultaneously.

The goal of this testing approach is to verify that the system remains responsive when users request sports information such as team standings, athlete statistics, or upcoming game schedules.

Performance testing helps determine whether the system:

- Maintains acceptable response times when multiple users access the system
- Handles concurrent requests for sports data without delays
- Avoids server overload during moderate traffic conditions
- Provides stable access to sports information pages

Representative scenarios include:

- Multiple users checking standings at the same time

- Several users accessing team or athlete pages simultaneously
- Users viewing game schedules during sports events

#### Usability Testing

Usability testing evaluates how easily users can navigate and interact with the platform.

Since the primary users of Tu Deporte Aquí are sports fans looking for quick access to match results, standings, and athlete information, the system interface must allow users to find information efficiently and without confusion.

Usability testing focuses on verifying whether the platform:

- Allows users to locate sports information quickly
- Presents data in a clear and understandable format
- Provides intuitive navigation between sections of the platform
- Supports efficient user interaction without unnecessary complexity

Typical usability scenarios include:

- Finding the standings for a specific team
- Viewing the schedule of upcoming games
- Accessing athlete statistics
- Navigating between teams, standings, and news sections

#### Expected Outcomes

By applying performance and usability testing during later development phases, the project aims to ensure that the Tu Deporte Aquí platform provides reliable and user-friendly access to sports information.

These testing approaches help establish:

- Responsive system behavior under normal user activity
- Clear and intuitive navigation for sports fans
- Consistent access to teams, athletes, schedules, and standings
- A positive overall user experience

# 4 Specific class topics

## 4.1 Test-Driven Development (TDD)

**Purpose**

This section describes how Test-Driven Development (TDD) principles support the reliability and transparency goals of Tu Deporte Aquí. TDD provides a structured approach to validating expected system behavior before implementation, aligning with the documentation-first philosophy of Milestone 1.

**Role of TDD in the Project**

TDD helps ensure that system behavior is clearly defined and verifiable before development begins. Since Tu Deporte Aquí operates with uncertain and potentially conflicting data sources, predictable behavior under different conditions is critical.

Key benefits include:

- Early clarification of expected behavior
- Reduced ambiguity in system logic
- Improved reliability through automated validation
- Safer future refactoring and iteration
- Alignment between documentation and implementation

**Application to Tu Deporte Aquí**

In this project, TDD would be particularly useful for validating:

**Data Reliability Logic**

- Conflict detection between multiple sources
- Timestamp freshness evaluation
- Handling of delayed or missing updates
- Confidence indicator behavior

**System Behavior Under Uncertainty**

- Graceful degradation when sources fail (example, displaying "Score unavailable" instead of partial data)
- Fallback behavior when data is incomplete
- Transparency messaging to users (example, marking games as "stale" when data cannot be guaranteed)
- Consistent handling of provisional states

#### User-Facing Behavior

- Correct display of reliability indicators
- Mobile-first information prioritization
- Status messaging consistency
- Error and degraded state communication

### Example TDD Workflow

A typical workflow would include:

1. Define expected behavior in documentation.
2. Write a failing test describing the behavior.
3. Implement the minimal logic required to pass the test.
4. Refactor while keeping tests passing.
5. Expand coverage for edge cases and reliability scenarios.

This approach ensures that behavior remains predictable even as the system evolves.

### Alignment with Milestone 1

During Milestone 1, no production code is implemented. However, defining TDD strategy early provides:

- Clear behavioral expectations
- Testable requirements
- Validation planning for later milestones
- Stronger reliability modeling

This preparation reduces future development risk and supports structured system evolution.

### TDD Walkthrough Example

To illustrate the practical application of TDD, this section presents a step by step evolution of a simple function called `formatScore`. This function converts raw score data into an user friendly display format while handling incomplete or invalid data.

The following iterations show how tests were written first, failing initially, and then the function was updated to pass each test sequentially.

#### Iteration 1: Basic Formatting

**Test 1: Expected behavior:**

```
expect(formatScore(80, 75)).toBe("80 - 75");
```

**Initial stub implementation (fails test):**

```
function formatScore() {
  return "";
}
```

**Updated implementation (passes test):**

```
function formatScore(homeScore, awayScore) {
  return `${homeScore} - ${awayScore}`;
}
```

#### Iteration 2: Handle Missing Home Score

**Test 2:**

```
expect(formatScore(null, 75)).toBe("Score unavailable");
```

**Updated implementation:**

```
function formatScore(homeScore, awayScore) {
  if (homeScore == null) {
    return "Score unavailable";
  }
  return `${homeScore} - ${awayScore}`;
}
```

#### Iteration 3: Handle Missing Away Score

**Test 3:**

```
expect(formatScore(80, undefined)).toBe("Score unavailable");
```

**Updated implementation:**

```
function formatScore(homeScore, awayScore) {
  if (homeScore == null || awayScore == null) {
    return "Score unavailable";
  }
  return `${homeScore} - ${awayScore}`;
}
```

#### Iteration 4: Handle Both Scores Missing

**Test 4:**

```
expect(formatScore(null, null)).toBe("Score unavailable");
```

**Result:** Test passes, no changes needed.

#### Iteration 5: Handle Zero Scores Correctly

[.inserted]#**Test 5:**

```
expect(formatScore(0, 0)).toBe("0 - 0");
```

**Result:** Test passes, no changes needed.

#### Final Implementation

```
function formatScore(homeScore, awayScore) {
  if (homeScore == null || awayScore == null) {
    return "Score unavailable";
  }
  return `${homeScore} - ${awayScore}`;
}
```

At each step, the function was modified minimally to pass the new test while keeping all previous tests passing.

### Conclusion

Test-Driven Development supports the core goals of Tu Deporte Aquí by promoting reliability, transparency, and predictable system behavior. Establishing TDD principles early ensures that future implementation remains aligned with documented expectations and real-world data constraints. === 4.2 BDD and BDD Tool Usage

### Purpose

This section describes how the **Tu Deporte Aquí** team applied currently applies Behavior-Driven Development (BDD) principles to derive acceptance criteria from user stories and personas, and how those acceptance criteria were transformed into specification examples using Gherkin. We document the process we followed during the planning phase to define, validate, and outline our planned approach test system behavior. Unlike Milestone 1, where BDD was primarily used for automating these examples using Cucumber planning, in future milestones. this milestone BDD is actively used to guide implementation and verify behavior through automated tests.

### From User Stories to Acceptance Criteria

We began with the continue to derive acceptance criteria from user stories documented in Section 2.2.1, which were written in the language of the domain and captured the needs of our personas: Sebastian (the superfan), Abiel (the coach), Andres (the reporter), (Section 2.2.1) and Chris (the engineer).

For each user story, we analyzed it to extract refine them into concrete examples of desired system behavior. behavioral scenarios. These examples became acceptance criteria, still scenarios are expressed in domain terms. We did this by asking questions such as:

- What specific situations might this user encounter?
- What would using the system need to show or do Given–When–Then structure and stored in those situations?
- What would count as a successful outcome? `.feature` files within the codebase.

Consider the following user story from Epic 1:

> As Sebastian, I want each score and update to show its source so that I can judge credibility when multiple channels exist.

From this story, we derived the following acceptance criteria:

- When a score comes from an official league source, it should be clearly labeled as such.
- When a score comes from a social media source, it should be labeled as "unverified" or "provisional".
- The source name (e.g., "BSN Official", "Twitter") should be visible next to the score.
- If multiple sources report the same score, the system may show multiple source attributions.

We define scenarios such as:

```
Scenario: Display valid score for live game
  Given a live game with home score 80 and away score 75
  When the game is processed
  Then the display should show "80 - 75"
```

These scenarios represent concrete, testable examples of expected system behavior.

### From Acceptance Criteria to Specification Examples

We then refined the acceptance criteria into concrete specification examples. These examples illustrate the desired behavior with specific inputs and expected outputs. They form the foundation of our BDD Current BDD Workflow in the Project

scenarios and are captured in the `.feature` files located in the `features/` directory of our repository.#

For the story above, we created the following specification example:#

Our current BDD workflow follows three steps:

#### 1. Scenario Definition (Specification Layer)

We define system behavior using Gherkin-style scenarios stored in the repository. These scenarios:

- describe behavior in domain language
- are understandable by developers and stakeholders
- serve as a reference for implementation

Example scenarios cover:

- score formatting
- missing score handling
- game status mapping (live, final, scheduled)
- stale data detection

#### 2. Implementation (Application Layer)

The behavior described in scenarios is implemented in the application code. For example:

- `formatScore()` handles score formatting
- `mapGameStatus()` determines game state
- `isGameStale()` evaluates data freshness

These functions represent the system behavior defined in the scenarios.

#### 3. Automated Validation (Test Layer)

Instead of using a dedicated BDD runner at this stage, we validate behavior using **Vitest**, a JavaScript testing framework.

Each scenario validates machine requirements R-MAC-9 (Graceful Degradation) and R-MAC-7 (Error Logging), and addresses is translated into automated tests located in:

- `utils/scoreUtils.test.js`
- `utils/scoreIntegration.test.js`

Example test:

```
expect(formatScore(80, 75)).toBe("80 - 75");
```

These tests directly verify the needs of Chris, outcomes described in the platform engineer. BDD scenarios.

### From Acceptance Criteria to Specification Examples

We then refined the acceptance criteria into concrete specification examples. These examples illustrate the desired behavior with specific inputs and expected outputs. They form the foundation of our BDD scenarios and are captured in the `.feature` files located in the `features/` directory of our repository.

For the story above, we created the following specification example.

### The Feature Files

We organized our scenarios into three feature files, each corresponding to a requirement category.

#### Example: A Real Scenario from `domain_requirements.feature`

Here is a complete scenario from our domain requirements feature file.

```
Scenario: [.removed]#Display source attribution in standings# [.inserted]#Conflicting
scores from different sources#
  Given [.removed]#Sebastian navigates to league standings# [.inserted]#a game between
"Cangrejeros" and "Vaqueros" is being played#
  [.removed]#And the standings data was last updated "5 minutes ago"#
  [.removed]#And the source is "BSN Official"#
  [.inserted]#And a social media post reports the score as "75-70" for Cangrejeros#
  [.inserted]#And the official league website later reports the score as "76-70" for
Cangrejeros#
  When [.removed]#the standings page loads# [.inserted]#Then the two reports are
considered conflicting#
  Then [.removed]#Sebastian should see a source icon next to each team's record#
[.inserted]#And the official league report takes precedence#
  [.removed]#And Sebastian can tap the icon to see the source and timestamp#
  [.removed]#And the source information should be consistent across all standings
entries# [.inserted]#And the conflicting social media report is recorded as
unofficial#
  [.inserted]#And fans should be made aware that a discrepancy existed#
```

This example is concrete and unambiguous. It uses domain terminology (standings, source, timestamp) and references our persona (Sebastian) to keep the focus on user value.

This scenario traces directly to domain requirements R-DOM-2 (Conflict Detection) and R-DOM-3 (Conflict Preservation), and to user story "As Sebastian, I want the platform to flag when two sources disagree on a score".

#### Example: A Real Scenario from `interface_requirements.feature`

```
Scenario: Show when data was last updated
  Given the game data was last updated "2 minutes ago"[.removed]# with away score 75#
  When the game [.removed]#is processed# [.inserted]#details page loads#
  Then [.removed]#the display should show "80 - 75"# [.inserted]#Sebastian should see
a "Last updated" timestamp near the score#
```

```
  [.inserted]#And the timestamp should show the time since last update#
  [.inserted]#And the timestamp should be in a human-readable format#
```

This scenario validates interface requirement R-INT-5 (Update on Domain Event) and machine requirement R-MAC-5 (Data Freshness), and addresses the user story "As Sebastian, I want to see when each item was last updated".

**Example: A Real Scenario from `machine_requirements.feature`**

```
Scenario: Continue operating when a source fails
  Given source "BSN API" returns a 500 Internal Server Error
  When the system processes requests for data
  Then the system should not crash
  And the system should continue serving data from remaining sources
  And the system should set a "partial data" flag for affected entities
  And the system should log the error with source name and timestamp
  And the system should return HTTP 200 to the user with available data
```

This scenario validates machine requirements R-MAC-9 (Graceful Degradation) and R-MAC-7 (Error Logging), and addresses the needs of Chris, the platform engineer.

**Traceability Matrix Mapping Between Scenarios and Tests**

The following matrix traces every scenario in our feature files to the requirements it validates and the user stories that motivated it. This matrix is maintained in our documentation ensure coverage and to help assess impact when requirements change. table illustrates how BDD scenarios map to implementation and automated tests:

| BDD Scenario | Feature File | Requirement IDs | User Story / Persona |
|---|---|---|---|
| "Conflicting scores from different sources" | `domain_requirements.feature` | R-DOM-2, R-DOM-3, R-DOM-1 | [.removed]## |
| "Provisional report becomes confirmed" | `domain_requirements.feature` | R-DOM-4, R-DOM-1 | [.removed]## |
| "Official correction after publication" | `domain_requirements.feature` | R-DOM-6, R-DOM-10 | Abiel: prioritize verified league results |
| "Outdated game information" | `domain_requirements.feature` | R-DOM-7 | Sebastian: see when each item was last updated |
| "Source becomes unavailable" | `domain_requirements.feature` | R-DOM-8 | Chris: standardized fallback behavior |
| "Correction without formal announcement" | `domain_requirements.feature` | R-DOM-2, R-DOM-1, R-DOM-3 | Sebastian: flag conflicting scores |
| "High demand for accurate information during playoffs" | `domain_requirements.feature` | R-MAC-3, R-MAC-4 | Sebastian: score pages load reliably during playoffs |

| BDD Scenario | Feature File | Requirement IDs | User Story / Persona |
|---|---|---|---|
| "Show when data was last updated" | `interface_requirements.feature` | R-INT-5, R-MAC-5 | Sebastian: see when each item was last updated |
| "Indicate stale data to the user" | `interface_requirements.feature` | R-DOM-7, R-MAC-9 | Sebastian: know if information might be outdated |
| "Show conflicting reports from different sources" | `interface_requirements.feature` | R-DOM-2, R-DOM-3 | Sebastian: platform flags conflicting scores |
| "Handle missing data gracefully" | `interface_requirements.feature` | R-MAC-9 | Sebastian: explanation when update is missing |
| "Label provisional data from unverified sources" | `interface_requirements.feature` | R-DOM-4, R-INT-9 | Andres: avoid presenting uncertain info as definitive |
| "Display source attribution in standings" | `interface_requirements.feature` | R-DOM-1, R-INT-9 | Sebastian: each score shows its source |
| "Communicate system-wide degraded state" | `interface_requirements.feature` | R-MAC-9 | Sebastian: clear messages when data is missing; Chris: standardized fallback behavior |
| "Show confidence level for uncertain data" | `interface_requirements.feature` | R-DOM-5 | Sebastian: simple reliability cue during major events |
| "Continue operating when a source fails" | `machine_requirements.feature` | R-MAC-9, R-MAC-7 | Chris: standardized fallback behavior |
| "Fall back to cached data during source unavailability" | `machine_requirements.feature` | R-MAC-9, R-MAC-5 | Chris: standardized fallback behavior |
| "Log conflicts for debugging and analysis" | `machine_requirements.feature` | R-MAC-6 | Chris: automated tests that simulate conflicting data; dashboards for consistency |
| "Prioritize critical endpoints under high traffic" | `machine_requirements.feature` | R-MAC-3, R-MAC-4 | Sebastian: score pages load reliably during playoffs; Chris: load and stress tests |

| BDD Scenario | Feature File | Requirement IDs | User Story / Persona |
|---|---|---|---|
| "Resume normal operation after source recovery" | `machine_requirements.feature` | R-MAC-8, R-MAC-9 | Chris: measure availability and recovery time |
| "Expose reliability metrics for monitoring" | `machine_requirements.feature` | R-MAC-8 | Chris: dashboards for accuracy, consistency, availability |
| "Retain correction history for audit purposes" | `machine_requirements.feature` | R-MAC-10, R-DOM-10 | Chris: audit trail for corrections |
| "Handle malformed data from a source" | `machine_requirements.feature` | R-MAC-7, R-MAC-9 | Chris: fault injection tests |
| "Recover from database connection failure" | `machine_requirements.feature` | R-MAC-12, R-MAC-9 | Chris: measure availability and recovery time |

Milestone 1 Traceability Matrix (detailed matrix above)

| BDD Scenario | Implementation | Test Verification |
|---|---|---|
| Display valid score | `formatScore()` | score formatting test |
| Missing score fallback | `formatScore()` | returns "Score unavailable" |
| Map game status | `mapGameStatus()` | status mapping tests |
| Detect stale data | `isGameStale()` | timestamp validation tests |

Milestone 2 Mapping Between Scenarios and Tests (simplified focus on implementation)

This mapping ensures that each scenario is both implemented and verifiable.

### Tool Usage

#### Gherkin (Specification)

We use Gherkin syntax to define behavior in `.feature` files. These files serve as:

- living documentation
- a shared understanding of expected behavior
- a bridge between requirements and implementation

#### Vitest (Execution)

We use **Vitest** to execute tests that validate the behavior described in scenarios.

Tests are executed using:

```
npm run test
```

Vitest runs all test files and reports pass/fail results for each behavior.

### Current Limitations

At this stage, we have designed our approach for automating these specification examples in future milestones. do not yet use a dedicated BDD tool. tool such as Cucumber to directly execute `.feature` files. Instead:

- `.feature` files define behavior
- Vitest tests validate that behavior programmatically

This approach allows us to keep our scenarios in Gherkin format (plain text) and link them to step definitions written in the same programming language we will use for implementation. This keeps the scenarios readable by non-technical stakeholders while enabling automation. maintain BDD principles while keeping the testing setup simple during early implementation.

#### Planned Workflow

Once implementation begins, we plan to follow this workflow:

1. **Feature files** (the `.feature` files we have already written, located in `features/`) will remain in the repository as living documentation. 2. **Step definitions** will be implemented as the corresponding functionality is developed. Each Gherkin step will be mapped to code that interacts with the system under test (via its API, UI, or directly with domain logic) and asserts expected outcomes. 3. **Test hooks** will be used to set up and tear down test data, ensuring scenarios are isolated and repeatable. 4. **Scenarios will be run** locally by developers during implementation, and also as part of the CI pipeline to catch regressions.

#### Planned CI/CD Integration

We plan to integrate Cucumber into our CI/CD pipeline as follows:

* On every pull request, the full test suite (including all Cucumber scenarios) will be executed automatically. * If any scenario fails, the build will be marked as failed, preventing merge until the issue is resolved. * Test reports will be generated and made visible in the pull request, so reviewers can see which scenarios passed or failed. * We will maintain a separate test environment where reliability scenarios (source failures, conflicting data) can be simulated safely.

#### Planned Test Data and Isolation

We recognize that many scenarios depend on specific data conditions (e.g., a game exists, a source reports a score). To keep scenarios reliable and independent, we plan to:

* Use a test database that is reset before each test run. * Use factories or fixtures to create the necessary domain objects (games, sources, reports) for each scenario. * For machine requirements (e.g., simulating source failures), we will use test doubles or mock servers that can be programmed to respond in specific ways.

**Current Status and Next Steps**

At this stage (Milestone 1), we have completed:

* Analysis of user stories to extract acceptance criteria * Transformation of acceptance criteria into Gherkin scenarios * Organization of scenarios into feature files in the `features/` directory * Creation of a traceability matrix linking scenarios to requirements * Planning of automation approach, tool selection, and CI/CD integration

In future milestones, we will:

* Implement step definitions as the corresponding features are developed * Set up the Cucumber test harness and integrate it with the build system * Run the scenarios against the evolving implementation * Refine scenarios as requirements become better understood

**Example: End-to-End Behavior Validation**

The following scenario:

```
Scenario: Display score unavailable when score is missing
  Given a live game with no score data
  When the game is processed
  Then the display should show "Score unavailable"
```

Is implemented and validated through:

- logic in `formatScore()`
- automated test asserting fallback behavior

This demonstrates how behavior flows from specification → implementation → validation.

**Conclusion**

BDD provided a structured way to move from user stories to concrete, testable specifications while keeping the language of the domain throughout. Even in the planning phase, writing scenarios helped us clarify requirements and identify edge cases. The In Milestone 2, BDD is no longer only a planning tool but an active part of development. The team feature files now serve as living documentation that bridges the gap between domain experts and developers. Once implementation begins, Cucumber will allow us to automate these scenarios, ensuring that uses:

- Gherkin scenarios to define expected behavior
- application logic to implement that behavior
- Vitest tests to verify correctness

This approach ensures that the system we build matches the needs identi fied in our analysis and documented in our requirements.# behavior remains:

- clear

- testable
- aligned with user needs

As the project evolves, these scenarios can be further integrated with dedicated BDD tools for direct execution, but they already serve as a strong foundation for behavior-driven testing in the current system.

## 4.3 Use of Debugging

**Purpose**

This section outlines the expected debugging practices that will support reliability and transparency in Tu Deporte Aquí. Debugging will help verify that the system correctly handles conflicting data, delayed stale updates, and source failures—key reliability concerns identified in our prior A3 submission. These scenarios represent the types of debugging sessions we expect to conduct during implementation.

**Role of Debugging**

Debugging provides visibility into data flows, especially when sources disagree or updates arrive late. It allows us to examine failure paths before they reach users.

Key contributions:

- Verify conflict detection logic logic: ensures inconsistent scores don't confuse fans.
- Confirm staleness thresholds trigger transparency indicators indicators: shows users how fresh the data is.
- Inspect state during partial source failures failures: helps maintain service even if some sources fail.
- Validate degraded mode messaging messaging: informs users when data is limited or unavailable.

**Debugging Environment**

All scenarios use:

- **Browser DevTools** for frontend inspection and network analysis
- **IDE debugger** for backend code examination
- **Debug console** for expression evaluation during breakpoints

**Expected Debugging Scenarios**

The following scenarios represent examples of debugging sessions we plan to conduct during implementation to validate core system behaviors. Each scenario identifies a reliability concern, describes the hypothesis we will test, and outlines which debugging techniques will help us verify the behavior.

**Scenario 1: Conflict Detection in Score Merging Display with Missing Data**

**Hypothesis**: `mergeGameData` might overwrite conflicting When one of the scores without flagging is missing, the disagreement.

*Planned system should display "Score unavailable" instead of an incorrect score, making sure fans are not mislead by incomplete data.

**Debugging** Session:

- Set A breakpoint was set inside `mergeGameData` at source comparison logic. `buildDisplayGame` before the return statement.
- Create A test case: official site (21–14) vs social media (24–14). case was executed with `homeScore = null` and `awayScore = 75`.
- Use The debugger was stepped into `isScoreComplete` to verify how completeness was evaluated.
- The **watch panel** was used to monitor `existingGame.score`, `newSourceData.score`, `sourcePriority`. inspect `homeScore`, `awayScore`, and `scoreComplete`.
- Step through Execution continued into `formatScore` to verify whether merging correctly stores conflicting values or silently overwrites.

**Expected Outcome**: This debugging will confirm whether conflict detection logic is it was called.

**Observed Outcome**:

- It was observed that `isScoreComplete` returned `false` when `homeScore` was `null`.
- As a result, `formatScore` was not executed.
- The value `"Score unavailable"` was correctly implemented. If conflicts are being silently overwritten, we will need assigned to implement a separate conflict tracking queue. `displayScore`.

**Scenario 2: Staleness Threshold Behavior**

**Hypothesis**: Data older than 15 10 minutes during live play should be marked `Delayed` `Stale` and shown in UI. fans want fresh data, older information should be marked stale.

*Debugging

**Planned** Session:

- Set conditional A breakpoint in `evaluateFreshness` (only when `game.status === "Live"`). was set inside `isStaleTimestamp`.
- Simulate 18-minute data freeze using browser DevTools network throttling. A test case was executed with a timestamp older than 20 minutes.
- Use **debug console** The **watch panel** was used to evaluate `Date.now() - game.lastUpdated.getTime()` monitor `lastUpdatedDate`, `now`, and verify it exceeds 900,000 ms. `diffMinutes`.
- Step into helper function through to confirm `StalenessLevel.DELAYED` is returned.

- Check API response contains `freshness: "Delayed"` and verify UI badge renders correctly.

**Expected Outcome**: This debugging will verify that staleness detection works end-to-end across backend evaluation, API response, and frontend rendering. If any layer fails `diffMinutes` exceeds `thresholdMinutes`. - The **debug console** was used to communicate staleness, we will identify evaluate `now - lastUpdatedDate` in milliseconds.

**Observed Outcome**:

- The calculated `diffMinutes` exceeded the breakpoint and fix it. threshold value.
- The function returned `true`, indicating stale data.

**Scenario 3: Source Outage and Degraded Mode Full Workflow Integration**

**Hypothesis**: When a source times out, the system falls back `buildDisplayGame` correctly integrates all utility functions to cached data and updates status board without crashing.

*Planned produce a consistent UI-ready object.

**Debugging** Session:

- Set A breakpoint in source fetcher `catch` block. was set at the beginning of `buildDisplayGame`.
- Inject 30-second timeout fault to simulate source outage. A complete game object was passed with valid scores, status, and timestamp.
- Use The debugger stepped into each helper function: `mapGameStatus`, `isScoreComplete`, and `isStaleTimestamp`.
- The **call stack inspection** stack** was inspected to trace propagation to coordinator and verify `getCachedGames` is invoked. execution flow across functions.
- Use The **watch panel** was used to confirm `sourceHealthStatus` and `fallbackTriggered` are updated correctly.
- Verify `setSystemStatus` is called with `DegradedState.SOURCE_UNAVAILABLE`.

*Expected monitor intermediate values.

**Observed** Outcome: This debugging will confirm that degraded mode activates cleanly without crashes

- Each helper function returned expected values.
- `scoreComplete` was `true`, and that cached data is served with transparency messaging. If any error handling is missing, we will identify the gap in `formatScore` produced the source fetcher or coordinator. correct formatted score.
- The final object contained consistent and expected values for all fields.

**Integration with Testing**

Debugging will complement automated tests during implementation:

- When reliability tests fail, we will debug to inspect intermediate state.

- We will explore edge cases difficult to capture in tests.
- We will validate behavior before writing acceptance tests.

**Alignment with Project Goals**

Defining debugging practices early establishes clear expectations for behavior examination during development. These scenarios directly support the reliability and transparency focus of our A3, enabling us to systematically verify that fault-injection and stress-testing practices are working as intended.

**Conclusion**

Debugging will be a critical tool for ensuring Tu Deporte Aquí behaves predictably under uncertainty. By examining conflict detection, staleness, and degraded mode during implementation, we will build confidence that reliability requirements are verifiable and reducible to testable code behavior.

# 5 AI Disclosure

**Overview**

During the development of the Tu Deporte Aquí documentation, the team made limited and responsible use of generative artificial intelligence tools as supplementary support resources. These tools were used to assist with organization, drafting clarity, and conceptual exploration, but they did not replace the team's own analysis, understanding, or decision-making. All final material included in this document reflects the work and judgment of the project members.

**Tools Used**

The primary tools used were publicly available generative AI systems, including ChatGPT (OpenAI) and DeepSeek. Only free or standard access versions were utilized, without any specialized training, plugins, or proprietary datasets. These tools were treated as general-purpose assistants similar to reference materials, writing aids, or brainstorming tools.

**Purpose of AI Assistance**

AI support was incorporated mainly during early and intermediate stages of documentation development. Its use focused on:

- Generating initial organizational structures for sections
- Suggesting ways to express technical ideas more clearly
- Assisting with refinement of explanations and terminology
- Providing alternative phrasings to improve readability
- Supporting brainstorming of possible approaches or considerations
- Helping ensure consistency in formatting across large sections of documentation

In some technical areas, AI tools were also used to produce preliminary draft text that served as a starting point for further development by the team.

**Human Ownership and Contribution**

All content was critically reviewed, edited, and refined by the Tu Deporte Aquí team members. The team provided the project context, interpreted requirements, evaluated suggestions, and made all final decisions regarding what material would be included.

AI outputs were never accepted without modification. Instead, they were treated as rough drafts or suggestions that required validation and adaptation. The intellectual responsibility for the documentation—including correctness, coherence, and alignment with project goals—remains entirely with the human authors.

The team ensured that each section accurately reflected their own understanding of the system, course concepts, and engineering principles rather than relying on automated generation alone.

### Verification and Quality Control

To maintain academic integrity and technical reliability, the team followed a verification process when using AI assistance. This included:

- Reviewing AI-generated explanations for logical consistency
- Cross-checking technical concepts against course materials and official documentation
- Confirming that terminology and assumptions matched the project context
- Correcting any inaccuracies, ambiguities, or unsupported claims
- Ensuring that final content remained aligned with project requirements and milestones

When AI suggestions were incomplete, unclear, or incorrect, they were either revised or discarded.

### Academic Integrity Considerations

The team recognizes that generative AI tools can produce convincing but inaccurate information. For this reason, AI was treated strictly as a support tool rather than an authoritative source. The responsibility for originality, accuracy, and academic honesty remained with the team at all times.

The final documentation represents original work developed by the project members, with AI serving only as a facilitator for efficiency and clarity.

### Reflection on Learning

The use of generative AI tools contributed positively to productivity by accelerating brainstorming and helping structure complex documentation. More importantly, it encouraged critical evaluation skills, since team members needed to assess whether generated content was correct, relevant, and appropriate.

This process reinforced an important engineering practice: tools can assist development, but professional judgment and accountability cannot be delegated. The team views AI as a supportive resource that enhances learning when used responsibly and thoughtfully.

### Conclusion

Generative AI tools were used in a limited, transparent, and supportive role throughout this project. The Tu Deporte Aquí team maintains full responsibility for the final documentation, its accuracy, and its alignment with course objectives.